



كلية هندسة الذكاء الاصطناعي
FACULTY OF ARTIFICIAL INTELLIGENCE ENGINEERING

الجامعة السورية الخاصة
SPU SYRIAN PRIVATE UNIVERSITY

Non-verbal communication system based on hand movement

(Graduation Project 2) prepared to obtain the degree of B.Sc. in the
Department of Artificial Intelligence Engineering and Data Science

Authors:

Sedra Najib

Supervised by:

Dr. Walaa Saffour

1447-1448 هـ
2025-2026 م

• Damascus

SUPERVISOR CERTIFICATION

I certify that the preparation of this project, entitled
“Project Title,”

Non-verbal communication system based on hand movement

prepared by

“Students’ Names,” sedra najib

was carried out under my supervision at the Faculty of Artificial Intelligence Engineering in partial fulfillment of the requirements for the degree of **B.Sc.** in the Department of Artificial Intelligence Engineering and Data Science.

Name:

Date:

Signature:

Abstract

Communication with patients who are unable to speak is a major challenge in healthcare environments. This project presents a smart non-verbal patient assistance system that enables patients to communicate their needs using hand gestures captured by a camera.

The system employs computer vision techniques through MediaPipe to detect and track hand landmarks in real time. By counting the raised fingers, predefined gestures are recognized and mapped to common patient requests such as emergency assistance, pain, water, food, doctor, family, bed adjustment, and boredom. Once a gesture is identified, the system automatically plays an audio notification, sends an alert to medical staff via Telegram, and stores the request in a local database for future review. A graphical user interface displays the current system status during operation.

The proposed system provides a simple, fast, and cost-effective solution that enhances communication between patients and healthcare providers without requiring wearable devices or physical contact. Its real-time performance and automatic request logging make it suitable for hospitals and healthcare facilities, improving patient safety and reducing response time.

Table of contents

Abstract.....	2
Chapter 1	7
Image and Video Processing	7
1.1.1 A SIMPLE IMAGE FORMATION MODEL	8
1.2 IMAGE SAMPLING AND QUANTIZATION.....	10
1.2.1 BASIC CONCEPTS IN SAMPLING AND QUANTIZATION.....	10
1.2.2 REPRESENTING DIGITAL IMAGES	13
1.2.3 LINEAR VS. COORDINATE INDEXING.....	18
1.2.4 SPATIAL AND INTENSITY RESOLUTION.....	19
1.2.5 IMAGE INTERPOLATION	21
1.3 SOME BASIC RELATIONSHIPS BETWEEN PIXELS	22
1.3.1 NEIGHBORS OF A PIXEL	22
1.3.2 ADJACENCY, CONNECTIVITY, REGIONS, AND BOUNDARIES.....	23
1.4 INTRODUCTION IN VIDEO PROCESSING	25
1.4.1 Digital Video.....	26
1.4.2 Sampling for digital image sequences.....	27
1.4.3 Quantization.....	27
1.4.4 Sampling for color video	28
1.4.5 Sampling analog video sources	28
Chapter 2	29
Hand Detection And Tracking.....	29
2.1 Architecture	30
2.2 Blaze Palm Detector	31
2.3 Hand Landmark Model.....	32
2.4 Dataset and Annotation	33
2.5 Results	34
2.6 Implementation in MediaPipe.....	35
2.7 Application examples	36
Chapter 3	38
3.1 Accessing the Camera Feed.....	39

3.2 Initializing Mediapipe Hands Model	39
3.3 Finger Counting Algorithm.....	39
3.4 Integrating into Main Loop.....	41
3.5 RESULTS AND DISCUSSION	43
3.6 APPLICATIONS.....	43
3.7 TECHNOLOGIES USED	44
3.8 IMPLEMENTATION.....	44
3.9 Hand DETECTION	44
3.10 ALGORITHM DEVELOPMENT.....	49
3.11 TESTING AND ANALYSIS	51
3.12 RESULT AND DISCUSSION.....	53
Chapter 4	57
Practical implementation and results.....	57
4.1 Introduction	58
4.2 Hand Detection and Command Execution Flow	58
4.3 Finger Status Evaluation and Counting Subsystem	74
4.4 Conducted experiments	77
References	84

Table of Figure:

Figure 1.1 An example of digital image acquisition.....	8
Figure 1.2 shows a continuous image f that we want to convert to digital form.....	10
Figure 1.3 shows a continuous image projected onto the plane of a 2D sensor.....	12
Figure 1.4 shows, we define the origin of an image at the top left corner.....	14
Figure 1.5 Coordinate convention used to represent digital images.....	15
Figure 1.6 An image exhibiting saturation and noise	16
Figure 1.7 shows the number of megabytes required to store square images	17
Figure 1.8 shows. The important thing to notice here is that each pixel is assigned a linear index value that identifies it uniquely.....	18
Figure 1.9 is the region itself. This border does not satisfy the definition of a closed path.....	24
Figure 2.1 Rendered hand tracking result.....	29
Figure 2.2 Palm detector model architecture.....	30
Figure 2.3 Architecture of our hand landmark model.....	31
Figure 2.4 Examples of four datasets.....	33
Figure 2.5 The hand landmark model's output controls when the hand detection model is triggered.....	35
Figure 2.6 Screenshots of real-time gesture recognition.....	36
Figure 2.7 Example of real-time AR effects based on our predicted hand skeleton.....	36
Figure 3.1 counting left hand finger.....	42
Figure 3.2 counting Both hand Finger.....	43
Figure 3.3 process of application.....	46
Figure 3.4 working diagram	47
Figure 3.5 Hand and Figure Detection.....	48
Figure 3.6 Hand Figure Count 1.....	48
Figure 3.7 Hand Finger Count 2.....	48
Figure 3.8 Hand and Figure Count 3.....	48
Figure 3.9 Hand Finger Count 4.....	48
Figure 3.10 Hand Finger Count 5.....	48
Figure 3.11 Hand Finger for 'All the Best'.....	49
Figure 3.12 Hand Finger Count 3.....	49
Figure 3.13 Flowchart for hand gesture recognition algorithm based on finger counting.....	50
Figure 3.14 Finger detection for four finger counts.....	52

Figure3.15 Hand gesture used in the experiment. (A and N uses one gesture to represent 1 and 5 finger counts, B to M uses four gestures to represent 2, 3 and 4 finger count).....	52
Figure3.16 Graphical flow chart for hand gesture recognition algorithm based on finger counting.....	53
Figure3.17 Percentage of successful hand gesture detection.....	54
Figure3.18 Maximum distance between fingers for four finger counts.....	54
Figure3.19 Effect of noise on the gesture recognition algorithm	55
Figure3.20 Effect of light intensity and size of a hand on the gesture recognition algorithm.....	55
Figure3.21 Effect of salt and pepper noise at 0.4 noise density on hand gesture recognition algorithm.....	56
Figure3.22 Effect of size on the hand gesture recognition algorithm.....	57
Figure3.23 Effect of the watch on the hand gesture recognition algorithm (a) the original image and (b) the binary image.....	57
Figure3.24 Effect of prayer thread on the hand gesture recognition algorithm (a) the original image and (b) the binary image.....	57
Figure4.1 Flowchart for Printing Commands	60
Figure4.2 Finger Counting flowchart	63
Figure4.3 patient assistant window	65
Figure4.4 clinical decision support (CDS) window.....	67

Chapter 1

Image and Video Processing

1.1 Digital image processing intro

Digital Image Processing (DIP) serves as the core foundation for modern computer vision and Human-Computer Interaction (HCI) systems. Before implementing advanced algorithms to track or recognize visual patterns, it is essential to understand how physical scenes are captured, digitized, and mathematically represented in computer memory

This chapter establishes the fundamental concepts of digital images, including matrix representation, sampling, quantization, and color models. For the proposed hand-movement non-verbal communication system, these basics are critical; tasks like isolating the hand from backgrounds, managing video resolution for real-time processing, and handling lighting variations all rely directly on these pixel-level fundamentals. [1]

1.1.1 A SIMPLE IMAGE FORMATION MODEL

As introduced in Section 1.1, we denote images by two-dimensional functions of the form $f(x, y)$. The value of f at spatial coordinates (x, y) is a scalar quantity whose physical meaning is determined by the source of the image, and whose values are proportional to energy radiated by a physical source (e.g., electromagnetic waves). As a consequence, $f(x, y)$ must be nonnegative† and finite; that is, [1]

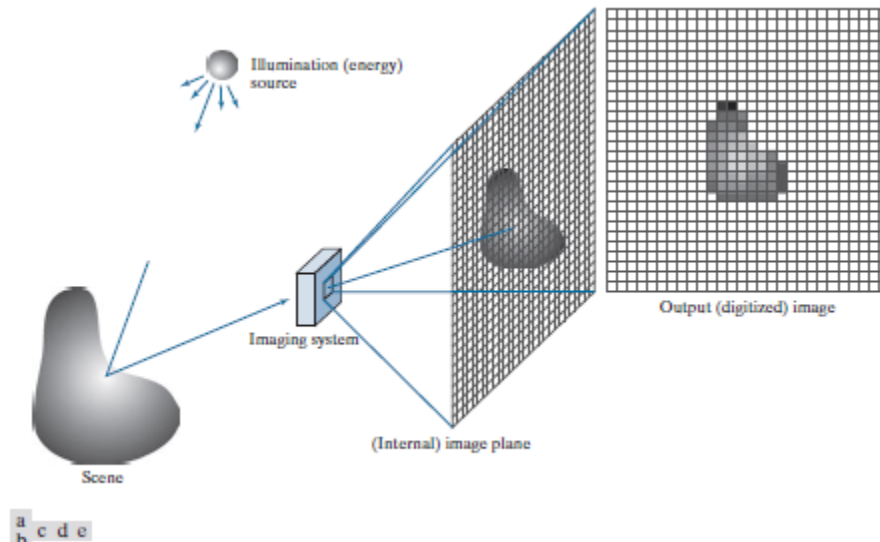


Figure 1.1

An example of digital image acquisition. (a) Illumination (energy) source.

(b) A scene. (c) Imaging system. (d) Projection of the scene onto the image plane. (e) Digitized image.

.

$$0 \leq f(x, y) < \infty \quad (1-1)$$

Function $f(x, y)$ is characterized by two components: (1) the amount of source illumination incident on the scene being viewed, and (2) the amount of illumination reflected by the objects in the scene. Appropriately, these are called the illumination and reflectance components, and are denoted by $i(x, y)$ and $r(x, y)$, respectively. The two functions combine as a product to form $f(x, y)$:

$$f(x, y) = i(x, y) r(x, y) \quad (1-2)$$

where

$$0 \leq i(x, y) < \infty \quad (1-3)$$

and

$$0 \leq r(x, y) \leq 1 \quad (1-4)$$

Thus, reflectance is bounded by 0 (total absorption) and 1 (total reflectance). The nature of $i(x, y)$ is determined by the illumination source, and $r(x, y)$ is determined by the characteristics of the imaged objects. These expressions are applicable also to images formed via transmission of the illumination through a medium, such as a chest X-ray. In this case, we would deal with a transmissivity instead of a reflectivity function, but the limits would be the same as in Eq. (1-4), and the image function formed would be modeled as the product in Eq. (1-2). **[1]**

1.2 IMAGE SAMPLING AND QUANTIZATION

As discussed in the previous section, there are numerous ways to acquire images, but our objective in all is the same: to generate digital images from sensed data. The output of most sensors is a continuous voltage waveform whose amplitude and spatial behavior are related to the physical phenomenon being sensed. To create a digital image, we need to convert the continuous sensed data into a digital format. This requires two processes: sampling and quantization. [1]

1.2.1 BASIC CONCEPTS IN SAMPLING AND QUANTIZATION

Figure 1.2 (a) shows a continuous image f that we want to convert to digital form. An image may be continuous with respect to the x - and y -coordinates, and also in

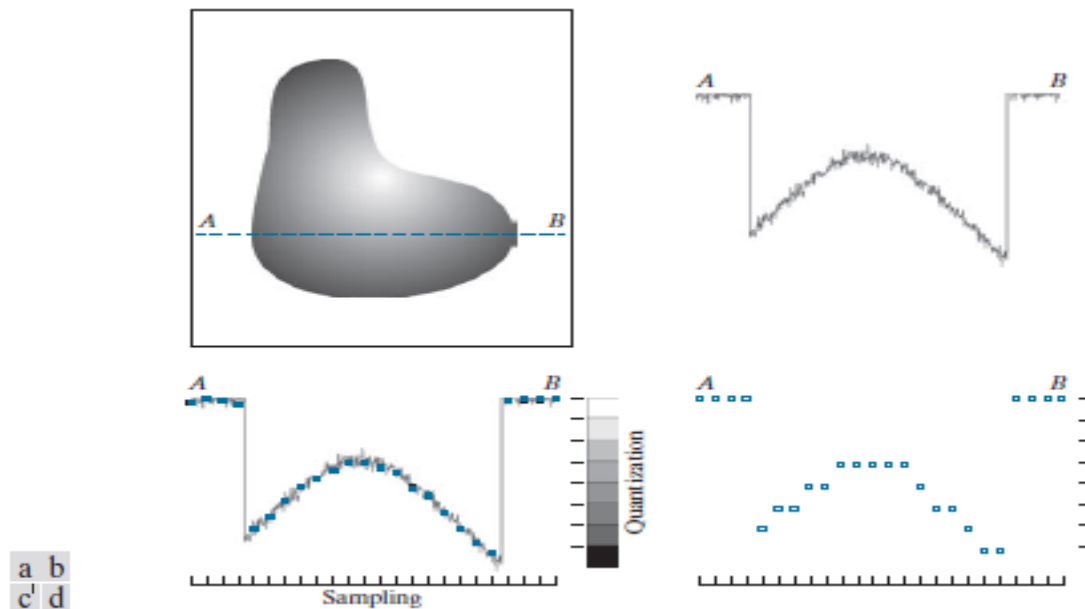


Figure 1.2

(a) Continuous image. (b) A scan line showing- intensity variations along line AB in the continuous image. (c) Sampling and quantization.(d) Digital scan line. (The black border in (a) is included for clarity. It is not part of the image).

amplitude. To digitize it, we have to sample the function in both coordinates and also in amplitude. Digitizing the coordinate values is called sampling. Digitizing the amplitude values is called quantization.

The one-dimensional function in Fig. 1.2(b) is a plot of amplitude (intensity level) values of the continuous image along the line segment AB in Fig. 1.2(a). The random variations are due to image noise. To sample this function, we take equally spaced samples along line AB, as shown in Fig. 1.2(c). The samples are shown as small dark squares superimposed on the function, and their (discrete) spatial locations are indicated by corresponding tick marks in the bottom of the figure. The set of dark squares constitute the sampled function. However, the values of the samples still span (vertically) a continuous range of intensity values. In order to form a digital function, the intensity values also must be converted (quantized) into discrete quantities. The vertical gray bar in Fig. 1.2(c) depicts the intensity scale divided into eight discrete intervals, ranging from black to white. The vertical tick marks indicate the specific value assigned to each of the eight intensity intervals. The continuous

intensity levels are quantized by assigning one of the eight values to each sample, depending on the vertical proximity of a sample to a vertical tick mark. The digital samples resulting from both sampling and quantization are shown as white squares in Fig. 1.2(d). Starting at the top of the continuous image and carrying out this procedure downward, line by line, produces a two-dimensional digital image. It is implied in Fig. 1.2 that, in addition to the number of discrete levels used, the accuracy achieved in quantization is highly dependent on the noise content of the sampled signal. **[1]**

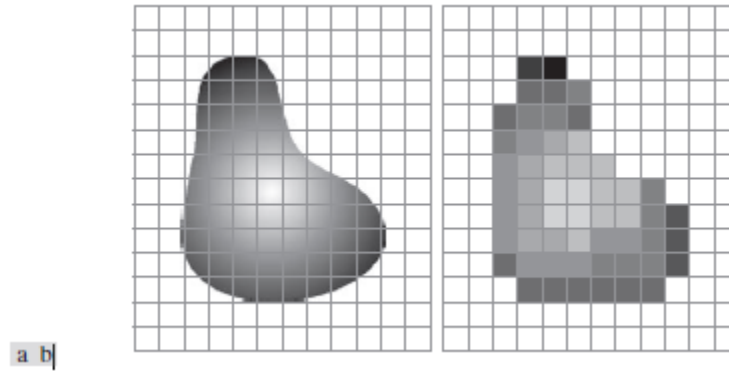


Figure 1.3

(a) Continuous image projected onto a sensor array. (b) Result of image sampling and quantization

In practice, the method of sampling is determined by the sensor arrangement used to generate the image. When an image is generated by a single sensing element combined with mechanical motion, the output of the sensor is quantized in the manner described above. However, spatial sampling is accomplished by selecting the number of individual mechanical increments at which we activate the sensor to collect data. Mechanical motion can be very exact so, in principle, there is almost no limit on how fine we can sample an image using this approach. In practice, limits on sampling accuracy are determined by other factors, such as the quality of the optical components used in the system.

When a sensing strip is used for image acquisition, the number of sensors in the strip establishes the samples in the resulting image in one direction, and mechanical motion establishes the number of samples in the other. Quantization of the sensor outputs completes the process of generating a digital image.

When a sensing array is used for image acquisition, no motion is required. The number of sensors in the array establishes the limits of sampling in both directions. Quantization of the sensor outputs is as explained above. Figure 1.3 illustrates this concept. Figure 1.3(a) shows a continuous image projected onto the plane of a 2D sensor. Figure 1.3 (b) shows the image after sampling and quantization. The quality of a digital image is determined to a large degree by the number of samples and discrete intensity levels used in sampling and quantization. However, as we will show later in this section, image content also plays a role in the choice of these parameters.[1]

1.2.2 REPRESENTING DIGITAL IMAGES

Let $f(s, t)$ represent a continuous image function of two continuous variables, s and t . We convert this function into a digital image by sampling and quantization, as explained in the previous section. Suppose that we sample the continuous image into a digital image, $f(x, y)$, containing M rows and N columns, where (x, y) are discrete coordinates. For notational clarity and convenience, we use integer values for these discrete coordinates: $x = 0, 1, 2, \dots, M-1$ and $y = 0, 1, 2, \dots, N-1$. Thus, for example, the value of the digital image at the origin is $f(0,0)$, and its value at the next coordinates along the first row is $f(0,1)$. Here, the notation $(0, 1)$ is used to denote the second sample along the first row. It does not mean that these are the values of the physical coordinates when the image was sampled. In general, the value of a digital image at any coordinates (x, y) is denoted $f(x, y)$, where x and y are integers. When we need to refer to specific coordinates (i, j) , we use the notation $f(i, j)$, where the arguments are integers. The section of the real plane spanned by the coordinates of an image is called the spatial domain, with x and y being referred to as spatial variables or spatial coordinates.

Figure 1.4 shows three ways of representing $f(x, y)$. Figure 1.4(a) is a plot of the function, with two axes determining spatial location and the third axis being the values of f as a function of x and y . This representation is useful when working with grayscale sets whose elements are expressed as triplets of the form (x, y, z) , where x and y are spatial coordinates and z is the value of f at coordinates (x, y) . We will work with this representation briefly in Section 2.6.

The representation in Fig. 1.4 (b) is more common, and it shows $f(x, y)$ as it would appear on a computer display or photograph. Here, the intensity of each point in the display is proportional to the value of f at that point. In this figure, there are only three equally spaced intensity values. If the intensity is normalized to the interval $[0,1]$, then each point in the image has the value 0, 0.5, or 1. A monitor or printer converts these three values to black, gray, or white, respectively, as in Fig1.4 (b). This type of representation includes color images, and allows us to view results at a glance.

As Fig. 1.4 (c) shows, the third representation is an array (matrix) composed of the numerical values of $f(x, y)$. This is the representation used for computer processing. In equation form, we write the representation of an $M * N$ numerical array as **[1]**

$$f(x,y) = \begin{bmatrix} f(0,0) & f(0,1) & \dots & f(0,N-1) \\ f(1,0) & f(1,1) & \dots & f(1,N-1) \\ \vdots & \vdots & & \vdots \\ f(M-1,0) & f(M-1,1) & \dots & f(M-1,N-1) \end{bmatrix} \quad (1.5)$$

The right side of this equation is a digital image represented as an array of real numbers. Each element of this array is called an image element, picture element, pixel, or pel. We use the terms image and pixel throughout the book to denote a digital image and its elements. Figure 1.5 shows a graphical representation of an image array, where the x - and y -axis are used to denote the rows and columns of the array.

Specific pixels are values of the array at a fixed pair of coordinates. As mentioned earlier, we generally use $f(i, j)$ when referring to a pixel with coordinates (i, j) .

We can also represent a digital image in a traditional matrix form:[1]

$$A = \begin{bmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,N-1} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,N-1} \\ \vdots & \vdots & & \vdots \\ a_{M-1,0} & a_{M-1,1} & \cdots & a_{M-1,N-1} \end{bmatrix} \quad (1.6)$$

Clearly $a_{ij} = f(i,j)$, so Eqs(1,5) and (1,6) denote identical arrays

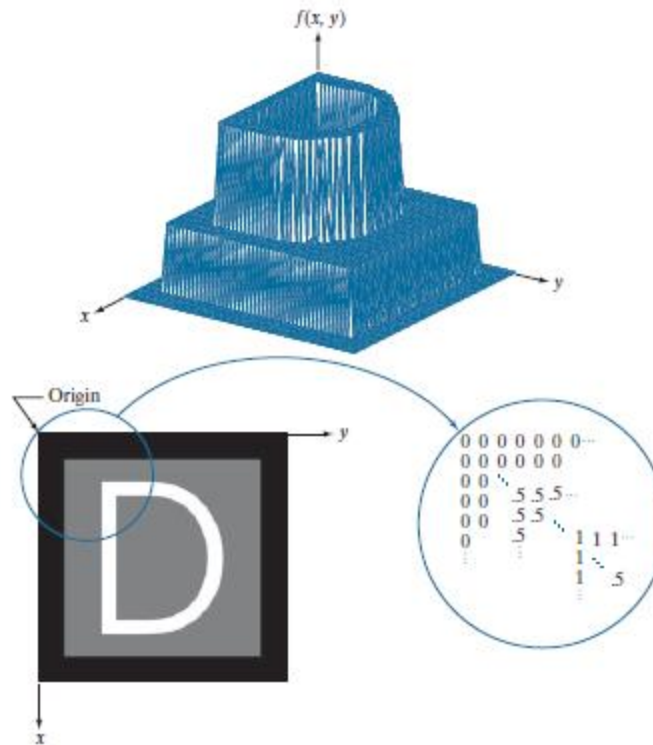


Figure 1. 4

(a) Image plotted as a surface. (b) Image displayed as a visual intensity array. (c) Image shown as a 2-D numerical array. (The numbers 0, .5, and 1 represent black gray, and white, respectively.)

As Fig. 1.4 shows, we define the origin of an image at the top left corner. This is a convention based on the fact that many image displays (e.g., TV monitors) sweep an image starting at the top left and moving to the right, one row at a time. More

important is the fact that the first element of a matrix is by convention at the top left of the array. Choosing the origin of $f(x, y)$ at that point makes sense mathematically because digital images in reality are matrices. In fact, as you will see, sometimes we use x and y interchangeably in equations with the rows (r) and columns (c) of a matrix.

It is important to note that the representation in Fig1.4, in which the positive x -axis extends downward and the positive y -axis extends to the right, is precisely the right-handed Cartesian coordinate system with which you are familiar,[†] but shown rotated by 90° so that the origin appears on the top, left.[1]

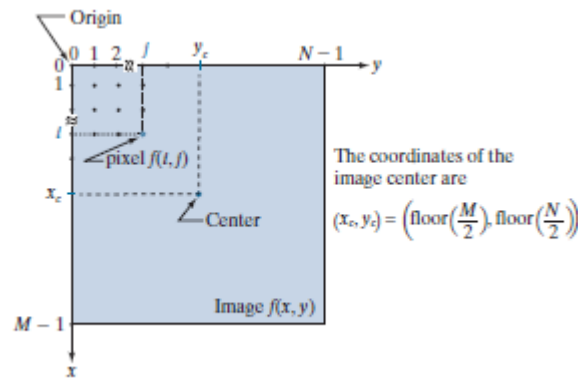


Figure1. 5

Coordinate convention used to represent digital images. Because coordinate values are integers, there is a one-to-one correspondence between x and y and the rows (r) and columns (c) of a matrix.

The *center* of an $M \times N$ digital image with origin at $(0,0)$ and range to $(M-1, N-1)$ is obtained by dividing M and N by 2 and rounding *down* to the nearest integer. This operation sometimes is denoted using the floor operator, floor , as shown in Fig. 1.5 This holds true for M and N even or odd. For example, the center of an image of size 1023×1024 is at $(511, 512)$. Some programming languages (e.g., MATLAB) start indexing at 1 instead of at 0. The center of an image in that case is found at $(x, y) = (M/2, N/2)$. $c = (\text{floor}(M/2) + 1, \text{floor}(N/2) + 1)$

To express sampling and quantization in more formal mathematical terms, let Z and R denote the set of integers and the set of real numbers, respectively. The sampling process may be viewed as partitioning the xy -plane into a grid, with the

coordinates of the center of each cell in the grid being a pair of elements from the Cartesian product Z^2 (also denoted $Z \times Z$) which, as you may recall, is the set of all ordered pairs of elements (z_i, z_j) with z_i and z_j being integers from set Z . Hence, $f(x, y)$ is a digital image if (x, y) are integers from Z^2 and f is a function that assigns an intensity value (that is, a real number from the set of real numbers, R) to each distinct pair of coordinates (x, y) . This functional assignment is the quantization process described earlier. If the intensity levels also are integers, then $R = Z$, and a digital image becomes a 2-D function whose coordinates and amplitude values are integers. This is the representation we use in the book.

Image digitization requires that decisions be made regarding the values for M , N , and for the number, L , of discrete intensity levels. There are no restrictions placed on M and N , other than they have to be positive integers. However, digital storage and quantizing hardware considerations usually lead to the number of intensity levels, L , being an integer power of two; that is

$$L = 2^k \quad (1-7)$$

where k is an integer. We assume that the discrete levels are equally spaced and that they are integers in the range $[0, L - 1]$. [1]

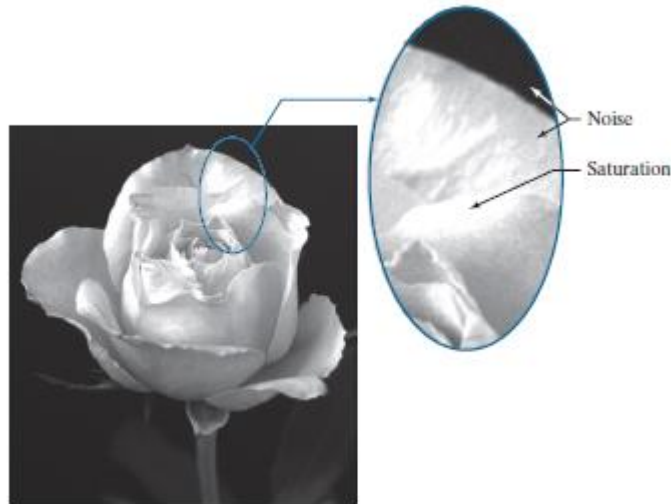


Figure 1. 6

An image exhibiting saturation and noise. Saturation is the highest value beyond which all intensity values are clipped (note how the entire saturated area has a high, constant intensity level). Visible noise in this case appears as a grainy texture pattern. The dark background is noisier, but the noise is difficult to see.

Sometimes, the range of values spanned by the gray scale is referred to as the *dynamic range*, a term used in different ways in different fields. Here, we define the

dynamic range of an imaging system to be the ratio of the maximum measurable intensity to the minimum detectable intensity level in the system. As a rule, the upper limit is determined by *saturation* and the lower limit by *noise*, although noise can be present also in lighter intensities. Figure 2.20 shows examples of saturation and slight visible noise. Because the darker regions are composed primarily of pixels with the minimum detectable intensity, the background in Fig. 2.20 is the noisiest part of the image; however, dark background noise typically is much harder to see. The dynamic range establishes the lowest and highest intensity levels that a system can represent and, consequently, that an image can have. Closely associated with this concept is *image contrast*, which we define as the difference in intensity between the highest and lowest intensity levels in an image. The *contrast ratio* is the ratio of these two quantities. When an appreciable number of pixels in an image have a high dynamic range, we can expect the image to have high contrast. Conversely, an image with low dynamic range typically has a dull, washed-out gray look. We will discuss these concepts in more detail in Chapter 3.[1]

The number, b , of bits required to store a digital image is

$$b = M * N * k \quad (1.8)$$

When $M = N$, this equation becomes

$$b = N^2 k \quad (1.9)$$

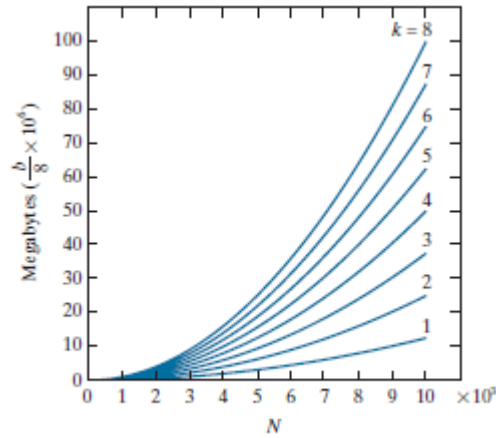


Figure 1.7

Number of megabytes required to store images for various values of N and k .

Figure 1.7 shows the number of megabytes required to store square images for various values of N and k (as usual, one byte equals 8 bits and a megabyte equals 106 bytes).

When an image can have 2^K possible intensity levels, it is common practice to

refer to it as a “ k -bit image,” (e.g., a 256-level image is called an *8-bit image*). Note that storage requirements for large 8-bit images (e.g., 10,000 * 10,000 pixels) are not insignificant. [1]

1.2.3 LINEAR VS. COORDINATE INDEXING

The convention discussed in the previous section, in which the location of a pixel is given by its 2-D coordinates, is referred to as coordinate indexing, or subscript indexing. Another type of indexing used extensively in programming image processing algorithms is linear indexing, which consists of a 1-D string of nonnegative integers based on computing offsets from coordinates (0,0). There are two principal types of linear indexing, one is based on a row scan of an image, and the other on a column scan. Figure 1.7 illustrates the principle of linear indexing based on a column scan.

The idea is to scan an image column by column, starting at the origin and proceeding down and then to the right. The linear index is based on counting pixels as we scan the image in the manner shown in Fig. 1.7. Thus, a scan of the first (leftmost) column yields linear indices 0 through $M - 1$. A scan of the second column yields indices M through $2M - 1$, and so on, until the last pixel in the last column is assigned the linear index value $MN - 1$. Thus, a linear index, denoted by a , has one of MN possible values: 0, 1, 2, ..., $MN - 1$, as Fig. 1.7 shows. The important thing to notice here is that each pixel is assigned a linear index value that identifies it uniquely.

The formula for generating linear indices based on a column scan is straightforward and can be determined by inspection. For any pair of coordinates (x, y) , the corresponding linear index value is [1]

$$a = My + x \quad (1.10)$$

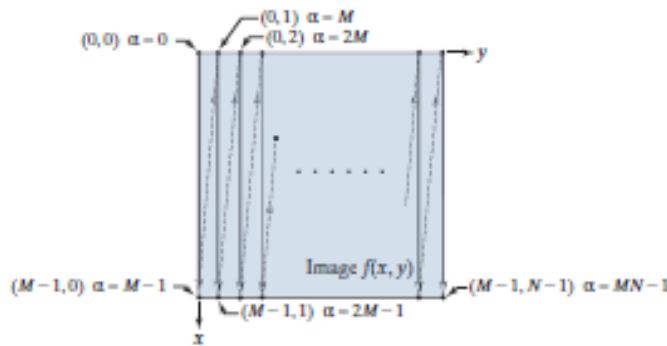


Figure 1. 1

Illustration of column scanning for generating linear indices Shown are several 2-D coordinates (in parentheses) and their corresponding linear indices Conversely, the coordinate indices for a given linear index value a are given by the equations†

$$x = a \bmod M \quad (1.11)$$

and

$$y = (a - x) M \quad (1.12)$$

Recall that $a \bmod M$ means “the remainder of the division of a by M .” This is a formal way of stating that row numbers repeat themselves at the start of every column.

Thus, when $a = 0$, the remainder of the division of 0 by M is 0, so $x = 0$. When $a = 1$, the remainder is 1, and so $x = 1$. You can see that x will continue to be equal to a until $a = M - 1$. When $a = M$ (which is at the beginning of the second column), the remainder is 0, and thus $x = 0$ again, and it increases by 1 until the next column is reached, when the pattern repeats itself. Similar comments apply to Eq. (1.2). See Problem 2.11 for a derivation of the preceding two equations.[1]
of a by M .”

1.2.4 SPATIAL AND INTENSITY RESOLUTION

Intuitively, spatial resolution is a measure of the smallest discernible detail in an image. Quantitatively, spatial resolution can be stated in several ways, with line pairs per unit distance, and dots (pixels) per unit distance being common measures.

Suppose that we construct a chart with alternating black and white vertical lines, each of width W units (W can be less than 1). The width of a line pair is thus $2W$, and there are $1/(2W)$ line pairs per unit distance. For example, if the width of a line is 0.1 mm, there are 5 line pairs per unit distance (i.e., per mm). A widely used definition of image resolution is the largest number of discernible line pairs per unit distance (e.g., 100 line pairs per mm). Dots per unit distance is a measure of image resolution used in the printing and publishing industry. In the U.S., this measure usually is expressed as dots per inch (dpi). To give you an idea of quality, newspapers are printed with a resolution of 75 dpi, magazines at 133 dpi, glossy brochures at 175 dpi, and the book page at which you are presently looking was printed at 2400 dpi.

To be meaningful, measures of spatial resolution must be stated with respect to spatial units. Image size by itself does not tell the complete story. For example, to say that an image has a resolution of $1024 * 1024$ pixels is not a meaningful statement without stating the spatial dimensions encompassed by the image. Size by itself is

helpful only in making comparisons between imaging capabilities. For instance, a digital camera with a 20-megapixel CCD imaging chip can be expected to have a higher capability to resolve detail than an 8-megapixel camera, assuming that both cameras are equipped with comparable lenses and the comparison images are taken at the same distance. Intensity resolution similarly refers to the smallest discernible change in intensity level. We have considerable discretion regarding the number of spatial samples (pixels) used to generate a digital image, but this is not true regarding the number of intensity levels. Based on hardware considerations, the number of intensity levels usually is an integer power of two, as we mentioned when discussing Eq. (2-11). The most common number is 8 bits, with 16 bits being used in some applications in which enhancement of specific intensity ranges is necessary. Intensity quantization using 32 bits is rare. Sometimes one finds systems that can digitize the intensity levels of an image using 10 or 12 bits, but these are not as common.

Unlike spatial resolution, which must be based on a per-unit-of-distance basis to be meaningful, it is common practice to refer to the number of bits used to quantize intensity as the “intensity resolution.” For example, it is common to say that an image whose intensity is quantized into 256 levels has 8 bits of intensity resolution. However, keep in mind that discernible changes in intensity are influenced also by noise and saturation values, and by the capabilities of human perception to analyze and interpret details in the context of an entire scene (see Section 2.1). The following two examples illustrate the effects of spatial and intensity resolution on discernible detail. Later in this section, we will discuss how these two parameters interact in determining perceived image quality.. [1]

1.2.5 IMAGE INTERPOLATION

Interpolation is used in tasks such as zooming, shrinking, rotating, and geometrically correcting digital images. Our principal objective in this section is to introduce interpolation and apply it to image resizing (shrinking and zooming), which are basically image resampling methods. Uses of interpolation in applications such as rotation and geometric corrections will be discussed in Section 2.6.

Interpolation is the process of using known data to estimate values at unknown locations. We begin the discussion of this topic with a short example. Suppose that an image of size 500 * 500 pixels has to be enlarged 1.5 times to 750 * 750 pixels. A simple way to visualize zooming is to create an imaginary 750 * 750 grid with the same pixel spacing as the original image, then shrink it so that it exactly overlays the original image. Obviously, the pixel spacing in the shrunken 750 * 750 grid will be less than the pixel spacing in the original image. To assign an intensity value to any point in the overlay, we look for its closest pixel in the underlying original image and assign the intensity of that pixel to the new pixel in the 750 * 750 grid. When intensities have been assigned to all the points in the overlay grid, we expand it back to the specified size to obtain the resized image.

The method just discussed is called nearest neighbor interpolation because it assigns to each new location the intensity of its nearest neighbor in the original image (see Section 2.5 regarding neighborhoods). This approach is simple but, it has the tendency to produce undesirable artifacts, such as severe distortion of straight edges. A more suitable approach is bilinear interpolation, in which we use the four nearest neighbors to estimate the intensity at a given location. Let (x, y) denote the coordinates of the location to which we want to assign an intensity value (think of it as a point of the grid described previously), and let $v(x, y)$ denote that intensity value. For bilinear interpolation, the assigned value is obtained using the equation

$$v(x, y) = ax + by + cxy + d \quad (1.13)$$

where the four coefficients are determined from the four equations in four

unknowns that can be written using the four nearest neighbors of point (x, y).

Bilinear interpolation gives much better results than nearest neighbor interpolation,

with a modest increase in computational burden. The next level of complexity is bicubic interpolation, which involves the sixteen

nearest neighbors of a point. The intensity value assigned to point (x, y) is obtained using the equation

$$v(x,y)=\sum_{i=0}^3 \sum_{j=0}^3 a_{ij} x^i y^j \quad (1.14)$$

The sixteen coefficients are determined from the sixteen equations with sixteen

unknowns that can be written using the sixteen nearest neighbors of point

(x, y) . Observe that Eq. (1-13) reduces in form to Eq. (1-14) if the limits of both

summations in the former equation are 0 to 1. Generally, bicubic interpolation does

a better job of preserving fine detail than its bilinear counterpart. Bicubic interpolation

is the standard used in commercial image editing applications, such as Adobe

Photoshop and Corel Photopaint.

Although images are displayed with integer coordinates, it is possible during processing

to work with subpixel accuracy by increasing the size of the image using

interpolation to “fill the gaps” between pixels in the original image.[1]

1.3 SOME BASIC RELATIONSHIPS BETWEEN PIXELS

In this section, we discuss several important relationships between pixels in a digital

image. When referring in the following discussion to particular pixels, we use lowercase

letters, such as p and q .[1]

1.3.1 NEIGHBORS OF A PIXEL

A pixel p at coordinates (x, y) has two horizontal and two vertical neighbors with coordinates

$$(x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1) \quad (1.15)$$

This set of pixels, called the 4-neighbors of p, is denoted $N_4(p)$.

The four diagonal neighbors of p have coordinates

$$(x + 1, y + 1), (x + 1, y - 1), (x - 1, y + 1), (x - 1, y - 1) \quad (1.16)$$

and are denoted $N_D(p)$. These neighbors, together with the 4-neighbors, are called the 8-neighbors of p, denoted by $N_8(p)$. The set of image locations of the neighbors of a point p is called the neighborhood of p. The neighborhood is said to be closed if it contains p. Otherwise, the neighborhood is said to be open. [1]

1.3.2 ADJACENCY, CONNECTIVITY, REGIONS, AND BOUNDARIES

Let V be the set of intensity values used to define adjacency. In a binary image, $V = \{1\}$ if we are referring to adjacency of pixels with value 1. In a grayscale image, the idea is the same, but set V typically contains more elements. For example, if we are dealing with the adjacency of pixels whose values are in the range 0 to 255, set V could be any subset of these 256 values. We consider three types of adjacency:

1. 4-adjacency. Two pixels p and q with values from V are 4-adjacent if q is in the set $N_4(p)$
2. 8-adjacency. Two pixels p and q with values from V are 8-adjacent if q is in the set $N_8(p)$
3. m -adjacency (also called *mixed adjacency*). Two pixels p and q with values from V are m -adjacent if
 - (a) q is in $N_4(p)$ or
 - (b) q is in $N_D(p)$ and the set $N_4(p) \cap N_4(q)$ has no pixels whose values are from V .

Mixed adjacency is a modification of 8-adjacency, and is introduced to eliminate the ambiguities that may result from using 8-adjacency. For example, consider the pixel arrangement in **Fig. 1.8(a)** and let $V = \{1\}$. The three pixels at the top of **Fig. 1.8(b)** show multiple (ambiguous) 8-adjacency, as indicated by the dashed lines. This ambiguity is removed by using m -adjacency, as in **Fig. 1.8(c)**. In other words, the center and upper-right diagonal pixels are not m -adjacent because they do not satisfy condition (b).

A digital path (or curve) from pixel p with coordinates (x_0, y_0) to pixel q with coordinates (x_n, y_n) is a sequence of distinct pixels with coordinates

$$(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$$

where points (x_i, y_i) and (x_{i-1}, y_{i-1}) are adjacent for $1 \leq i \leq n$. In this case, n is the length of the path. If $(x_0, y_0) = (x_n, y_n)$ the path is a closed path. We can define 4-, 8-, or m -paths, depending on the type of adjacency specified. For example, the paths in **Fig. 1.8(b)** between the top right and bottom right points are 8-paths, and the path in **Fig. 1.8(c)** is an m -path.

Let S represent a subset of pixels in an image. Two pixels p and q are said to be connected in S if there exists a path between them consisting entirely of pixels in S . For any pixel p in S , the set of pixels that are connected to it in S is called a connected component of S . If it only has one component, and that component is connected, then S is called a connected set.

Let R represent a subset of pixels in an image. We call R a region of the image if R is a connected set. Two regions, R_i and R_j are said to be adjacent if their union forms a connected set. Regions that are not adjacent are said to be disjoint. We consider 4- and 8-adjacency when referring to regions. For our definition to make sense, the type of adjacency used must be specified. For example, the two regions of 1's in **Fig. 1.8(d)** are adjacent only if 8-adjacency is used (according to the definition in the previous [1])

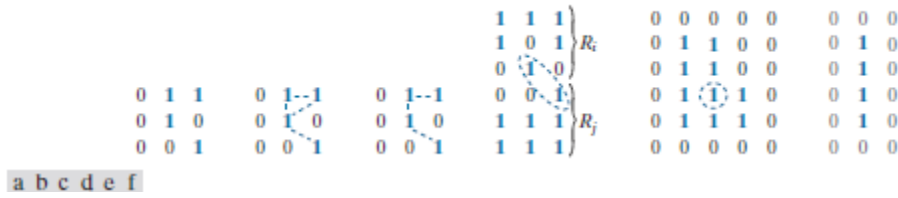


Figure1. 2

(a) An arrangement of pixels. (b) Pixels that are 8-adjacent (adjacency is shown by dashed lines). (c) m-adjacency. (d) Two regions (of 1's) that are 8-adjacent. (e) The circled point is on the boundary of the 1-valued pixels only if 8-adjacency between the region and background is used. (f) The inner boundary of the 1-valued region does not form a closed path, but its outer boundary does.

paragraph, a 4-path between the two regions does not exist, so their union is not a connected set).

Suppose an image contains K disjoint regions, R_k , $k=1, 2, \dots, K$, none of which touches the image border.† Let R_u denote the union of all the K regions, and let $(R_u)^c$

denote its complement (recall that the complement of a set A is the set of

points that are not in A). We call all the points in R_u the foreground, and all the

points in $(R_u)^c$ the background of the image. The boundary (also called the border or contour) of a region R is the set of pixels in R that are adjacent to pixels in the complement of R

. Stated another way, the border of a region is the set of pixels in the region that have at least one background neighbor. Here again, we must specify the connectivity being used to define adjacency. For example, the point circled in **Fig. 1.8(e)** is not a member of the border of the 1-valued region if 4-connectivity is used between the region and its background, because

the only As a rule, adjacency between points in a region and its background is defined using

8-connectivity to handle situations such as this possible connection between that point and the background is diagonal..

The preceding definition sometimes is referred to as the inner border of the

region to distinguish it from its outer border, which is the corresponding border in

the background. This distinction is important in the development of border-following

algorithms. Such algorithms usually are formulated to follow the outer boundary

in order to guarantee that the result will form a closed path. For instance, the inner

border of the 1-valued region in **Fig. 1.8(f)** is the region itself. This border does not

satisfy the definition of a closed path. On the other hand, the outer border of the

region does form a closed path around the region.

If R happens to be an entire image, then its boundary (or border) is defined as the set of pixels in the first and last rows and columns of the image. This extra definition is required because an image has no neighbors beyond its border. Normally, when we refer to a region, we are referring to a subset of an image, and any pixels in the boundary of the region that happen to coincide with the border of the image are included implicitly as part of the region boundary.

The concept of an edge is found frequently in discussions dealing with regions and boundaries. However, there is a key difference between these two concepts. The boundary of a finite region forms a closed path and is thus a “global” concept. As we will discuss in detail in Chapter 10, edges are formed from pixels with derivative values that exceed a preset threshold. Thus, an edge is a “local” concept that is based on a measure of intensity-level discontinuity at a point. It is possible to link edge points into edge segments, and sometimes these segments are linked in such a way that they correspond to boundaries, but this is not always the case. The one exception in which edges and boundaries correspond is in binary images. Depending on the type of connectivity and edge operators used (we will discuss these in Chapter 10), the edge extracted from a binary region will be the same as the region boundary. [1]

1.4 INTRODUCTION IN VIDEO PROCESSING

Image sequence processing refers to methods for the digital processing of a set of images with related content. Often the image sequence results from the observation of a time-varying scene (video). Since the range of techniques which fall into the category of image sequence processing is extremely broad, the topic is approached in three main areas:

1. Fundamental video concepts.

The image formation process, three-dimensional motion, projected motion, analog and digital video, sampling and quantization are discussed.

2. Motion estimation.

The problem of motion estimation is central to image sequence processing. The characteristics of, and difficulties associated with the motion estimation problem are discussed, followed by a detailed description of three popular motion estimation frameworks.

3. Applications.

Digital video compression, an application of image sequence processing techniques, has made practical numerous new and emerging video and entertainment technologies. These compression technologies rely heavily on motion estimation and prediction, in addition to more traditional coding techniques

The discussion on compression is followed by an introduction to an emerging area in image sequence restoration { multi-frame super-resolution restoration. These techniques extend the classical single-image restoration theory by integrating information from multiple frames, taking advantage of the motion occurring in the image sequence. The chapter is concluded with a brief overview of other important applications. [2]

1.4.1 Digital Video

Digital video refers to representations of the time-varying image information which utilize sampling and quantization. The resulting digital representation may be conveniently stored, manipulated, copied and processed using digital computers with no loss of fidelity. Digitizing time-varying imagery does, however, result in massive quantities of data generated at extremely high bit-rates. This raises important issues regarding the representation (coding and compression), processing, indexing, and transmission of image sequences, all of which are active research and application areas.

Uncoded or raw digital video is represented by spatio-temporal samples of the time-varying, continuous image $f(x; y; t)$ projected on the image plane. A digital image sequence (video) consists of a temporal sequence of frames (or less commonly fields) which in turn consist of a finite, regular 2-D lattice of picture elements (called pels or pixels) which are samples of the spatially-varying illumination intensity pattern incident at the focal plane. Individual pixels may have one or more components to represent multi-spectral information (e.g. three RGB color components). Pixel values are quantized and stored using a finite bit-length digital representation. Typically from 1 to 24 bits are used to represent each pixel value. As a result of the flexibility available in choosing the spatio-temporal sampling pattern as well as in the representation of pixel values, a dizzying variety of digital video formats exist. A very approachable introduction to digital video may be found . [2]

1.4.2 Sampling for digital image sequences

The optical system projects a time-varying, continuous-valued image $f(x; y; t)$ of the scene onto a 2-D focal plane. The value of $f(x; y; t)$ is recorded at discrete locations to produce a sampled representation of the visual information. The sampled representation $f(n_1; n_2; k)$ is indexed by the discrete-valued variables $n_1 \in \{0, 1, \dots, N_1\}$, $n_2 \in \{0, 1, \dots, N_2\}$ and $k \in \{0, 1, \dots, K\}$. The spatio-temporal sampling pattern is typically highly regular and constitutes a sampling grid [8]. The sampling points in the spatial and temporal dimensions must be carefully chosen so as to faithfully represent the spatio-temporal variation of $f(x; y; t)$ without requiring an excessive number of sampling points which would lead to redundancy. Nyquist's sampling theorem [5, 8] provides a lower bound on the sampling grid density required to enable accurate representation of a signal with given spatial and temporal bandwidth. Sampling in accordance with Nyquist's theorem ensures that it is possible to reconstruct the original signal exactly provided that appropriate reconstruction filters are implemented.

Since real-world scenes are usually not spatially band-limited, low-pass, so-called anti-alias filters are often used to ensure that the instantaneous image is spatially band-limited, thereby enabling spatial sampling without artifacts. The sampling density in the temporal dimension is typically driven by the application and little can be done about the problem of temporal aliasing.

In modern imaging systems, area scan sensors such as CCD or CMOS arrays have replaced older scanning devices like the Vidicon tube. Area scan devices typically sample the entire image area over a single temporal integration period. This differs substantially from older scanning technologies where samples along a scan line derived from temporally distinct instants. As a result, there is a tendency away from interlaced capture and display systems. Most computer display systems and many camera systems are now able to function in progressive scan mode, yielding high quality frames free from motion artifacts. For in-depth discussions on sampling structures for spatio-temporal signals, [2]

1.4.3 Quantization

The process of sampling produces a continuous-valued function on a discrete sampling grid. To enable digital storage, transmission or processing of the sampled data the continuous-valued signal must be discretized to yield a finite bit-length representation. This process is known as quantization. A quantizer maps a continuous variable to a discrete variable which takes on values from a finite set of reconstruction levels. The mapping commonly takes the form of a non-decreasing staircase function with a finite number of transitions with corresponding reconstruction levels. Quantization is therefore an inherently lossy process. Quantizer design often requires the minimization of some distortion measure, such as mean squared or mean absolute quantization error. The design of the quantizer has a direct impact on the storage and transmission requirements for samples. For this reason, the design of quantizers plays an important role in compression. For further details on the characteristics and design of quantizers. [2]

1.4.4 Sampling for color video

The sampling of multi-band image sequences (including color video) raises additional issues regarding the sampling pattern used for each spectral band. Though direct color component sampling such as RGB is common, often the sampling patterns used for color video take advantage of characteristics of the human visual system. By representing color information in a luma/chroma color space rather than in color component form, it is possible to utilize reduced spatial sampling rates for the chrominance components since the human visual system has lower sensitivity to chrominance spatial frequencies as compared with similar luminance spatial frequencies. For additional information on color spaces and sampling patterns for the luma/chroma representation of color images, The most commonly used color sub-sampled digital image formats include 4:2:2 (the horizontal sampling rate is halved for chroma components) and 4:2:0 (the horizontal and vertical sampling rates are halved for chroma components). In the 4:4:4 format there is no sub-sampling of the chroma components. [2]

1.4.5 Sampling analog video sources

Sampling of analog video signals to yield digital video sequences is also possible and is routinely accomplished using video capture cards which interface with a computer system. It is important to realize, however, that this approach is fundamentally limited by the scanning characteristics of the analog video signal. If, for example, the analog video signal is interlaced, the resulting sampled output will have the same interlaced structure. Similarly the resolution limitations of the analog signal are reflected in the sampled output. In order to overcome these limitations, techniques for video filtering, enhancement, restoration and deinterlacing have been developed. [2]

Chapter 2

Hand Detection And Tracking

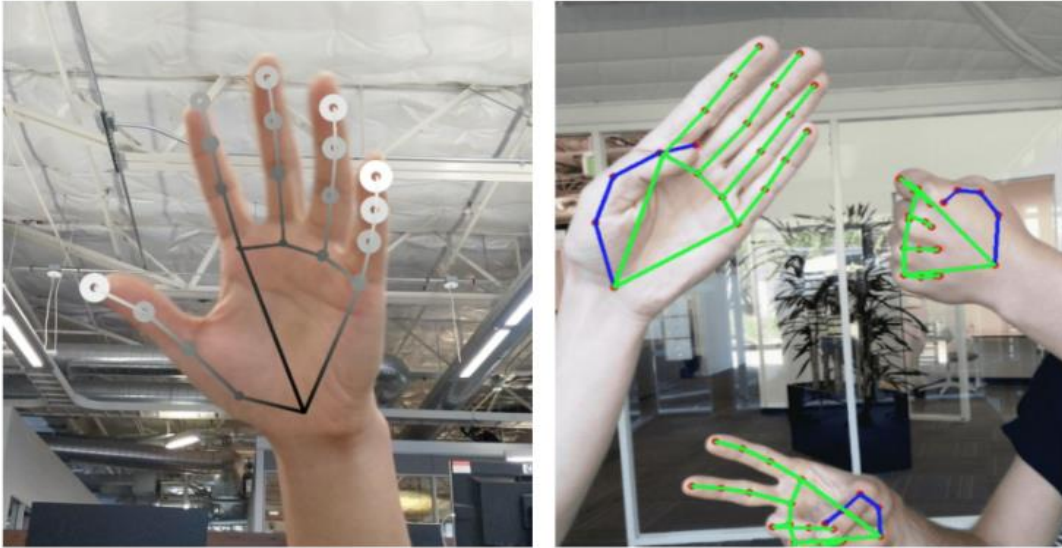


Figure 2.1

Rendered hand tracking result. (Left): Hand landmarks with relative depth presented in different shades. The lighter and larger the circle, the closer the landmark is towards the camera. (Right): Real-time multi-hand tracking on Pixel 3.

2.1 Architecture

Our hand tracking solution utilizes an ML pipeline consisting of two models working together:

- A palm detector that operates on a full input image and locates palms via an oriented hand bounding box.
- A hand landmark model that operates on the cropped hand bounding box provided by the palm detector and returns high-fidelity 2.5D landmarks.

Providing the accurately cropped palm image to the hand landmark model drastically reduces the need for data augmentation (e.g. rotations, translation and scale) and allows the network to dedicate most of its capacity towards landmark localization accuracy. In a real-time tracking scenario we derive a bounding box from the landmark prediction of the previous frame as input for the current frame, thus avoiding applying the detector on every frame. Instead, the detector is only applied on the first frame or when the hand prediction indicates that the hand is lost. [3]

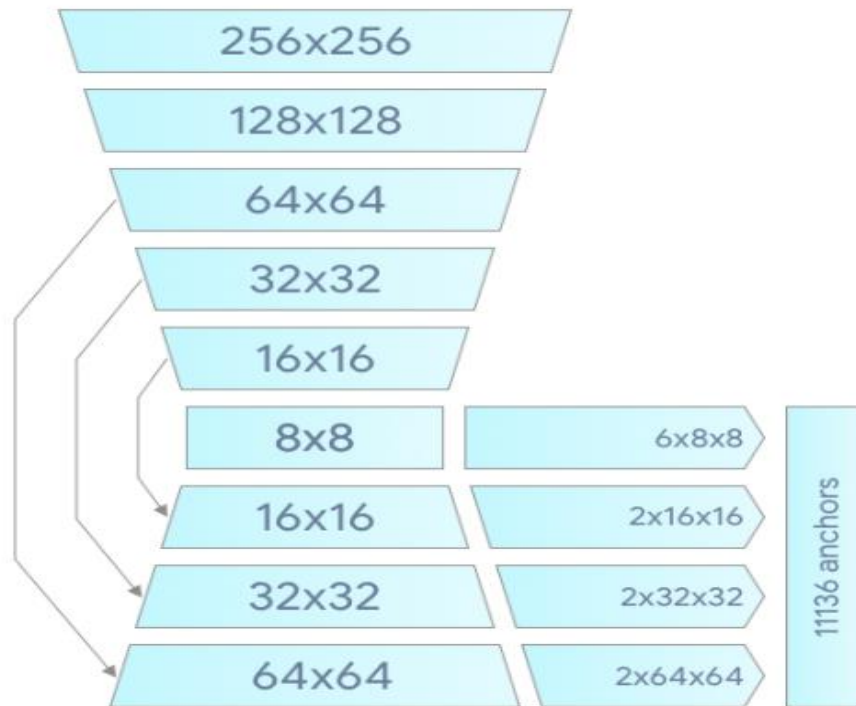


Figure2.2: Palm detector model architecture.

2.2 Blaze Palm Detector

To detect initial hand locations, we employ a singleshot detector model optimized for mobile real-time application similar to BlazeFace which is also available in MediaPipe. Detecting hands is a decidedly complex task: our model has to work across a variety of hand sizes with a large scale span ($\sim 20\times$) and be able to detect occluded and self-occluded hands. Whereas faces have high contrast patterns, e.g., around the eye and mouth region, the lack of such features in hands makes it comparatively difficult to detect them reliably from their visual features alone. Our solution addresses the above challenges using different strategies.

First, we train a palm detector instead of a hand detector, since estimating bounding boxes of rigid objects like

palms and fists is significantly simpler than detecting hands with articulated fingers. In addition, as palms are smaller objects, the non-maximum suppression algorithm works well even for the two-hand self-occlusion cases

, like handshakes. Moreover, palms can be modelled using only square bounding boxes ignoring other aspect ratios, and therefore reducing the number of anchors by a factor of 3_5. Second, we use an encoder-decoder feature extractor similar to FPN for a larger scene-context awareness even for small objects. Lastly, we minimize the focal loss during training to support a large amount of anchors resulting

from the high scale variance. High-level palm detector architecture. is shown in Figure 2.2 We present an ablation study of our design elements in Table 1. [3]

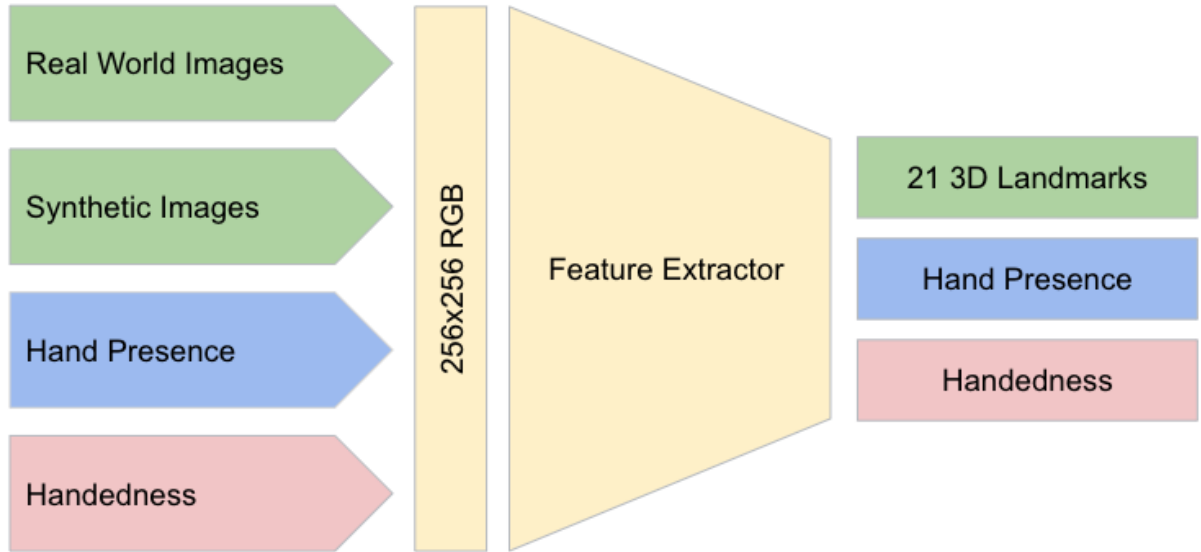


Figure 2.3:

Architecture of our hand landmark model. The model has three outputs sharing a feature extractor. Each head is trained by correspondent datasets marked in the same color. See Section 2.2 for more detail.

2.3 Hand Landmark Model

After running palm detection over the whole image, our subsequent hand landmark model performs precise landmark localization of 212.5D coordinates inside the detected hand regions via regression . The model learns a consistent internal hand pose representation and is robust even to partially visible hands and self-occlusions. The model has three outputs (see Figure 3):

1. 21 hand landmarks consisting of x, y, and relative depth.
2. A hand flag indicating the probability of hand presence in the input image.
3. A binary classification of handedness, e.g. left or right hand

We use the same topology as for the 21 landmarks The 2D coordinates are learned from both real-world images as well as synthetic datasets as discussed below with the relative depth w.r.t. the wrist point being learned only from synthetic images. To recover from tracking failure, we developed another output of the model similar tofor producing the probability of the event that a reasonably aligned hand is indeed present in the provided crop. If the score is lower than a threshold then the detector is triggered to reset tracking. Handedness is another important attribute for effective interaction using hands in AR/VR. This is

especially useful for some applications where each hand is associated with a unique functionality. Thus we developed a binary classification head to predict whether the input hand is the left or right hand. Our setup targets real-time mobile GPU inference, but we have also designed lighter and heavier versions of the model to address CPU inference on the mobile devices lacking proper GPU support and higher accuracy requirements of accuracy to run on desktop, respectively.[3]

2.4 Dataset and Annotation

To obtain ground truth data, we created the following datasets addressing different aspects of the problem:

- In-the-wild dataset: This dataset contains 6K images of large variety, e.g. geographical diversity, various lighting conditions and hand appearance. The limitation of this dataset is that it doesn't contain complex articulation of hands.
- In-house collected gesture dataset: This dataset contains 10K images that cover various angles of all physically possible hand gestures. The limitation of this dataset is that it's collected from only 30 people with limited variation in background. The in-the-wild and in-house dataset are great complements to each other to improve robustness
- Synthetic dataset: To even better cover the possible hand poses and provide additional supervision for depth, we render a high-quality synthetic hand model over various backgrounds and map it to the corresponding 3D coordinates. We use a commercial 3D hand model that is rigged with 24 bones and includes 36 blendshapes, which control fingers and palm thickness. The model also provides 5 textures with different skin tones. We created video sequences of transformation between hand poses and sampled 100K images from the videos. We rendered each pose with a random high-dynamic-range lighting environment and three different cameras. See Figure 4 for examples.

For the palm detector, we only use in-the-wild dataset, which is sufficient for localizing hands and offers the highest variety in appearance. However, all datasets are used for training the hand landmark model. We annotate the realworld images with 21 landmarks and use projected groundtruth 3D joints for synthetic images. For hand presence, we select a subset of real-world images as positive examples and sample on the region excluding annotated hand regions as negative examples. For handedness, we annotate a subset of real-world images with handedness to provide such data. [3]

2.5 Results

or the hand landmark model, our experiments show that the combination of real-world and synthetic datasets provides the best results. See Table 2 for details. We evaluate only on real-world images. Beyond the quality improvement, training with a large synthetic dataset leads to less jitter visually across frames. This observation leads us to believe that our real-world dataset can be enlarged for better generalization. Our target is to achieve real-time performance on mobile devices. We experimented with different model sizes and found that the “Full” model (see Table 3) provides a good. [3]

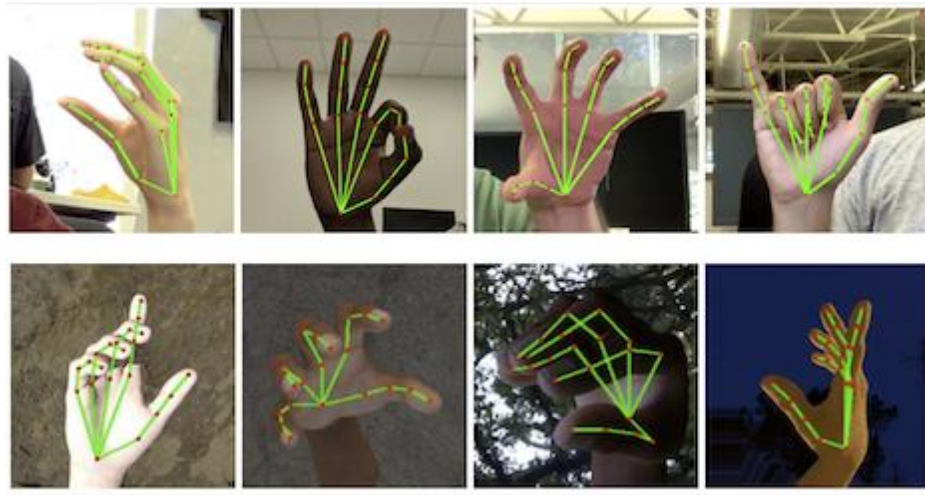


Figure 2.4: Examples of our datasets. (Top): Annotated realworld images. (Bottom): Rendered synthetic hand images with ground truth annotation. See Section 3 for details.

Model Variation	Average Precision
No decoder + cross entropy loss	86.22%
Decoder + cross entropy loss	94.07%
Decoder + focal loss	95.7%

Table 1: Ablation study of palm detector design elements of palm detector.

Dataset	MSE normalized by palm size
Only real-world	16.1%
Only synthetic	25.7%
Combined	13.4%

Table 2: Results of our model trained from different datasets.

trade-off between quality and speed. Increasing model capacity further introduces only minor improvements in quality but decreases significantly in speed (see Table 3 for details). We use the TensorFlow Lite GPU backend for ondevice inference [3]

Model	Params (M)	MSE	Time(ms) Pixel 3	Time(ms) Samsung S20	Time(ms) iPhone11
Light	1	11.83	6.6	5.6	1.1
Full	1.98	10.05	16.1	11.1	5.3
Heavy	4.02	9.817	36.9	25.8	7.5

Table 3: Hand landmark model performance characteristics.

2.6 Implementation in MediaPipe

With MediaPipe our hand tracking pipeline can be built as a directed graph of modular components, called Calculators. Mediapipe comes with an extensible set of Calculators to solve tasks like model inference, media processing, and data transformations across a wide variety of devices and platforms. Individual Calculators like cropping rendering and neural network computations are further optimized to utilize GPU acceleration. For example, we employ TFLite GPU inference on most modern phones. Our MediaPipe graph for hand tracking is shown in Figure 5. The graph consists of two subgraphs one for hand detection and another for landmarks computation. One key optimization MediaPipe provides is that the palm detector only runs as needed (fairly infrequently), saving significant computation. We achieve this by deriving the hand location in the current video frames from the computed hand landmarks in the previous frame, eliminating the need to apply the palm detector on every frame. For robustness, the hand tracker model also outputs an additional scalar capturing the confidence that a hand is present and reasonably aligned in the input crop. Only when the confidence falls below a certain threshold is the hand detection model reapplied to the next frame.[3]

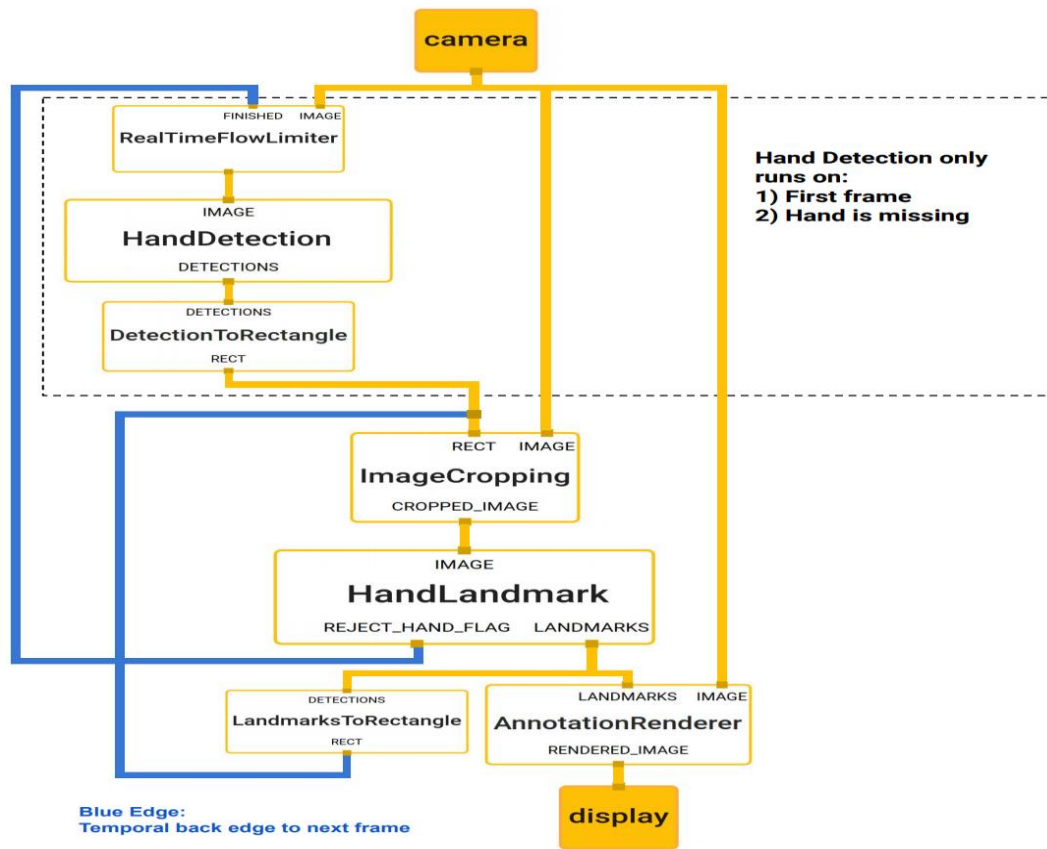


Figure 2.5

The hand landmark models output controls when the hand detection model is triggered. This behavior is achieved by MediaPipes powerful synchronization building blocks, resulting in high performance and optimal throughput of the ML pipeline

2.8 Application examples

Our hand tracking solution can readily be used in many applications such as gesture recognition and AR effects. On top of the predicted hand skeleton, we employ a simple algorithm to compute gestures, see Figure 6. First, the state of each finger, e.g. bent or straight, is determined via the accumulated angles of joints. Then, we map the set of finger states to a set of predefined gestures. This straightforward yet effective technique allows us to estimate basic static gestures with reasonable quality. Beyond static gesture recognition, it is also possible to use a sequence of landmarks to predict dynamic gestures. Another application is to apply AR effects on top of the skeleton. Hand based AR effects currently enjoy high popularity. In Figure 7, we show an example AR rendering of the hand skeleton in neon light style. [3]

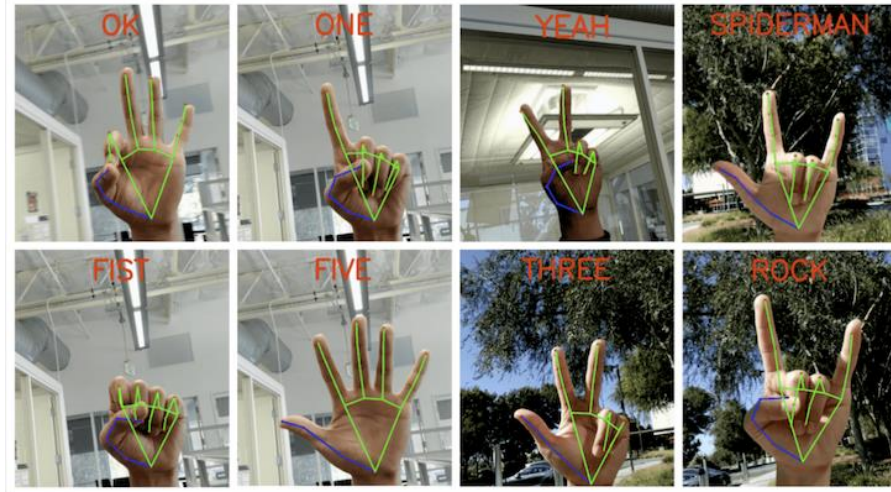


Figure 2.6

Screenshots of real-time gesture recognition. Semantics of gestures are rendered at top of the images.

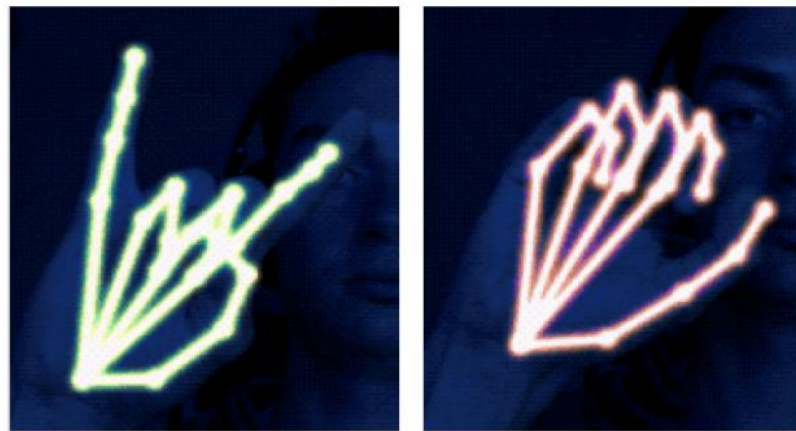


Figure 2.7:

Example of real-time AR effects based on our predicted hand skeleton.

, an end-to-end hand tracking solution that achieves real-time performance on multiple platforms. Our pipeline predicts 2.5D landmarks without any specialized hardware and thus, can be easily deployed to commodity devices. We open sourced the pipeline to encourage researchers and engineers to build gesture control and creative AR/VR applications with our pipeline [3]

Chapter 3

Hand Gesture

Recognition and Finger

Count

3.1 Accessing the Camera Feed

The first step involves accessing live video from a standard camera (such as a laptop's integrated camera or a USB camera). We initialized a VideoCapture object in OpenCV to start capturing video signals. Each frame from the camera is then processed individually. [4]

```
import cv2

import mediapipe as mp

# Initialize camera

cap = cv2.VideoCapture(0) # 0 denotes the primary camera

if not cap.isOpened():

    print("Error accessing camera.")

    cap.release()

    cv2.destroyAllWindows()
```

- Here, we create a VideoCapture instance and check whether the camera is opened successfully.
- If not, we release resources and destroy all windows safely

3.2 Initializing Mediapipe Hands Model

To perform hand landmark detection, we initialized Mediapipe's Hands pipeline. We configured it to track maximum two hands with a minimum confidence of 0.7.

```
mp_hands = mp.solutions.hands

hands = mp_hands.Hands(max_num_hands=2,

    min_detection_confidence=0.7,

    min_tracking_confidence=0.7)

mp_draw = mp.solutions.drawing_utils
```

- Here, Hands is initialized with maximum two hands and minimum confidence
- scores. The drawing_utils lets us visualize landmarks afterwards. [4]

3.3 Finger Counting Algorithm

For counting raised fingers, we first need to know which landmarks correspond to fingertips and knuckles. Tip IDs for thumb, index, middle, ring, and little finger are 4, 8, 12, 16, and 20, respectively.

```
def count_fingers(hand_landmarks, handedness):  
    """Count raised fingers given landmarks and handedness."""  
    finger_tips = [4, 8, 12, 16, 20]  
    thumb_tip = 4  
    thumb_base = 2  
    count = 0  
    landmarks = hand_landmarks.landmark  
    if handedness == 'Right':  
        if landmarks[thumb_tip].x < landmarks[thumb_base].x:  
            count += 1  
        else:  
            if landmarks[thumb_tip].x > landmarks[thumb_base].x:  
                count += 1  
    for tip in finger_tips[1:]:  
        if landmarks[tip].y < landmarks[tip-2].y:  
            count += 1  
    return count
```

- The algorithm checks thumb direction first and then for other four fingertips.
- If a finger's tip is above its knuckle, it is raised [4]



Figure 3.1 Counting Left hand finger.

3.4 Integrating into Main Loop

We combine all these components to process each video frame in real time.

```
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.flip(frame, 1) # mirror view
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = hands.process(rgb)
    if results.multi_hand_landmarks and results.multi_handedness:
        for hand, handedness in zip(results.multi_hand_landmarks, results.multi_handedness):
            mp_draw.draw_landmarks(frame, hand, mp_hands.HAND_CONNECTIONS)
            label = handedness.classification[0].label
            finger_count = count_fingers(hand, label)
            if label == 'Left':
```

```

pos = (10, 200)
else:
pos = (10, 150)
cv2.putText(frame, f'{label} Hand Fingers: {finger_count}',
pos,
cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
cv2.imshow('Hand Finger Count', frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
break

```

Here we:

- Flip the frame for mirror view.
- Convert color to RGB for Mediapipe processing.
- Detect hands and draw landmarks.
- Count raised fingers for each hand and display the results. [4]



Figure 3.2 Counting Both hand Fingers

3.5 RESULTS AND DISCUSSION

The developed system was tested on a standard laptop equipped with an ordinary CPU and webcam.

Performance Metrics:

- Frame Rate: Approximately 30–40 frames per second
- Detection Accuracy: Around 95% under adequate lighting
- Latency: Minimal and suitable for interactive use

The system consistently detected and tracked both hands in real time. Finger counting remained accurate when the hand was clearly visible and properly illuminated.

Minor inaccuracies occurred in cases of:

- Poor lighting conditions
- Rapid hand movements
- Excessive proximity to the camera

Despite these limitations, the system demonstrated strong stability and responsiveness. The lightweight architecture confirms that effective gesture recognition can be achieved without specialized hardware. [4]

3.6 APPLICATIONS

A Hand Gesture Recognition System recognizes the Shapes and or orientation depending on implementation to task the system into performing some job. Gestures is a form of non-verbal information. A person can make numerous gestures at a time. As humans through vision perceive human gestures and for computer we need a camera, it is a subject of great interest for computer vision researchers such as performing an action based on gestures of the person.

A. Virtual presence:

Sometimes in a situation like machine, electricity failure, emergency hostiles condition or some remote areas which are inaccessible to humans, it could very dangerous for human operators to be physically appear to operate the machines or inthe working conditions

B. Bomb disposal:

Minedisposal is safer when human beings are replaced by the robot arm which will work on the same concept of this system. It leads to decrease in the risk of life of a human and it also inspires the efficient handling of the state.[5]

3.7 TECHNOLOGIES USED

CV2 MODULE OpenCV

(Open Source Computer Vision) is a library of programming functions mainly aimed at present-time computer vision. First developed by Intel, it was later supported by Willow Garage then OpenCV (which was later developed by Intel). OpenCV also supports the deep learning frameworks.

NUMPY MODULE It

is not another programming language but a Python addition unit. It provides fast and capable operations on arrays of homogeneous data. It covers Python into a high-level language for working with mathematical data, like MATLAB.

MATLAB MODULE In

MATLAB, we can use packages to make the maintenance and organization of MATLAB files easier. They help to create a namespace for the specific set of files contained within them. Packages are special files that can cover class folders, files, functions, and class definition files, and other letters. [5]

3.8 IMPLEMENTATION

The first move is to capture the copy from camera and to define an area of interest in the surround, it is important as the image can contain a lot of variables and these variables can result in unwanted results and the data that needs to be processed is cheap to a large amount. To capture the image a web-camera is used that continuously captures surrounds and is used to get the raw data for giving out. The input picture we have here is uint8. The procured image is RGB and must be processed before that is before-processed before the workings are divided and response is made.

B) Facts Pre-Processing:

Pre-processing method can be completed 2-steps process:

- Divided
- Morphological clarifying

First Process is the Divided in process. It is done to change over grey-scale picture into the binary picture so we can have just two areas of interest in photo. That is, one will be hand and another one is related.

There are two main approaches to segmentation:

1) Pixel-based or local methods having: -

- a. Edge detection
- b. Boundary detection.

2) Region-based approach: -

- a. The region merging
- b. The region splitting
- c. Threshold method[5]

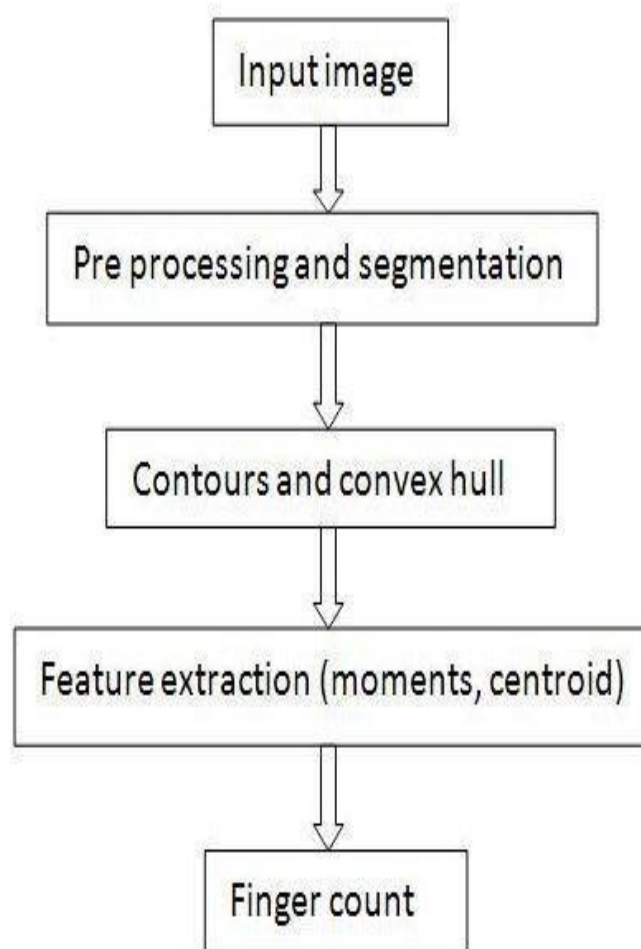


Figure3.3 Process of Application

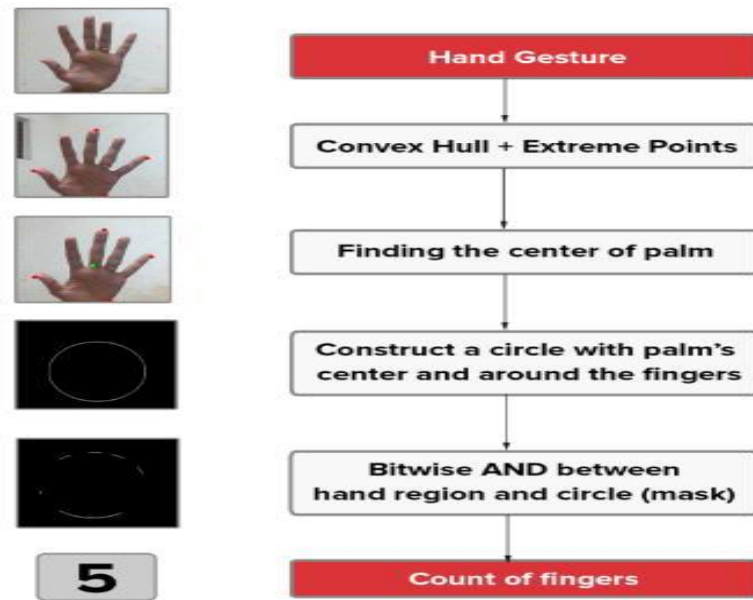


Figure3.4 Working Diagram

3.9 HAND DETECTION

Contours are the curves joining all the continuous points along the limit, having same color or intensity. The contours are a useful tool for shape analysis and object detection and recognition. The contour is drawn along the boundary of the hand image which is found after Threshold[5]

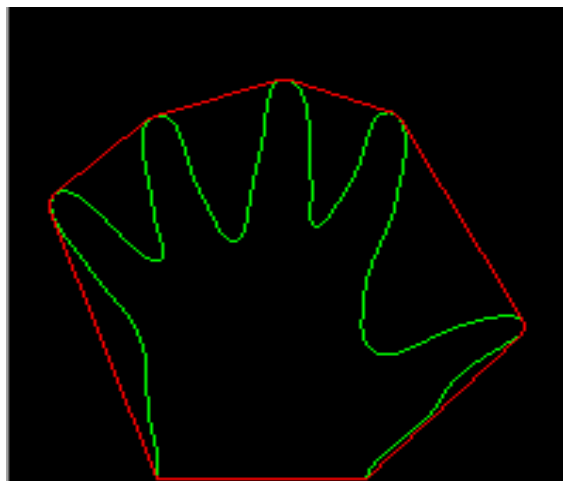


Figure3.4 Hand Detection

Convex Hull and Convexity Defects We now find the convex points and the fault points. The convex points are generally, the tip of the fingers. But there is other convex point too. So, we find convexity faults, which is the secret point of deviance on the contour. By this we can find the number of fingers extended and then we can perform different functions according to the number of fingers extended [5]

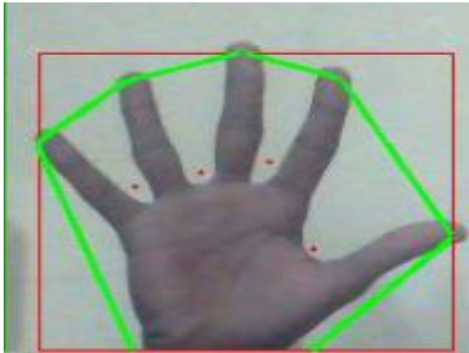


Figure 3.5 Hand and Finger Detection



Figure 3.8 Hand and Finger Count 3



Figure 3.6 Hand Figure Count 1



Figure 3.9 Hand Finger Count 4



Figure 3.7 Hand Finger Count 2



Figure 3.10 Hand Finger Count 5



Figure 3.11 Hand Finger for ‘All the Best’

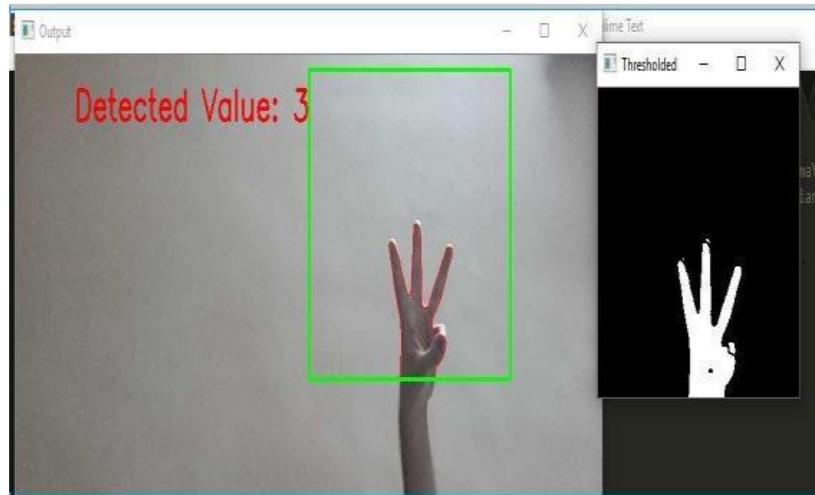


Figure3.12 Hand Finger Count 3

1. Background of pictures should be plain to get exact analysis of recognition of gestures.
 1. Extra random keep an eye on minutes is helpful for checking whether the forms of the layout picture and the picture of individual have same shape.
 2. For maintaining the performance, the database should contain the template images of small dimension.
- observe the following output by implementing algorithm [5]

3.10 ALGORITHM DEVELOPMENT

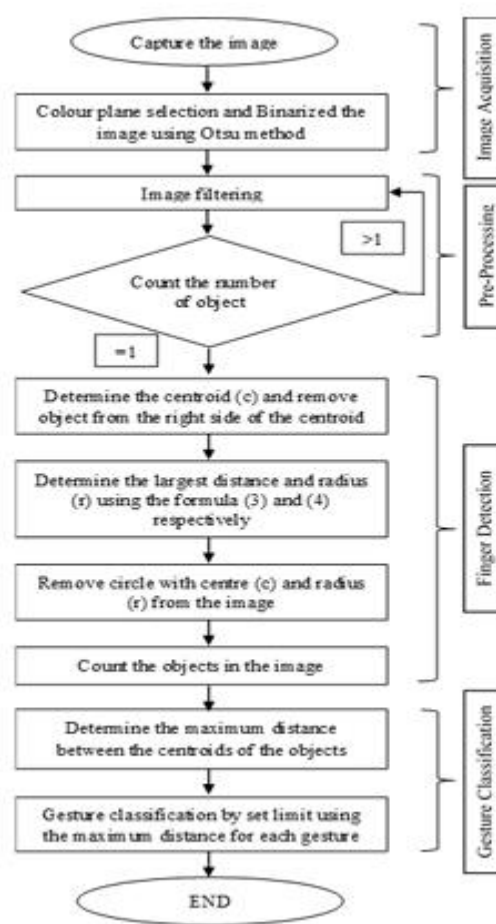


Figure 3.13 Flowchart for hand gesture recognition algorithm based on finger counting

A. Image Acquisition

Image acquisition is the first step in this system. Images are captured by an off-the-shelf HD webcam with a resolution of 16Mp. This webcam is interfaced with MATLAB 2012 to capture the image. The subject is requested to place their hand on a dark surfaced platform to capture the image. Fourteen hand gesture is used in this research which are obtained from one hand, and the subjects are asked to show the gesture in front of the camera to capture the image. Then, the image is load in the MATLAB 2012 software, and the image is resized half of its original size for the convenient of the user.

After that, the RGB image is converted into YCbCr color space. YCbCr color space is a YUV color space family, and it is also a linear luminance or chromaticity color space. YCbCr stands for Y is luminance, Cb and Cr are chromaticities of blue and red colors. The Cb and Cr are two-dimensional independents. The formula to convert an RGB color space to YCbCr color space is: [6]

$$\begin{bmatrix} Y \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 0.2990 & 0.5870 & 0.1440 \\ -0.1687 & -0.3313 & 0.5000 \\ 0.5000 & -0.4187 & -0.0813 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix} \quad (1)$$

Then, the image is converted into binary image using Otsu's method. Otsu's method is used to automatically perform clustering-based image thresholding or the reduction of a gray level image to a binary image. In other words, it is simply an algorithm for segmentation of object regions. A simple but effective tool to separate objects from the background. Otsu method principle one simply defines the within-class variance. The threshold value will be determined by the algorithm using Equation (2). Therefore, the user does not have to determine the threshold value manually. [6]

$$\sigma^2_{Within}(T) = \omega_B(T)\sigma^2_B + \omega_0(T)\sigma^2_0(T) \quad (2)$$

$$\begin{aligned} \text{where: } \omega_B(T) &= \sum_{i=0}^{T-1} p(i), \quad [0, L-1] \text{ is the range of intensity} \\ \omega_0(T) &= \sum_{i=0}^{L-1} p(i) \\ \sigma^2_B(T) &= \text{Variance of the pixels in the background (below threshold)} \\ \sigma^2_0(T) &= \text{Variance of the pixels in the foreground (above threshold)} \end{aligned}$$

B. Pre-Processing

After a successful binary image is determined, the image should be cleaned to process further. First, the small objects in the image must be removed. Thus, objects which are less than a certain pixel must be removed to obtain the object that the user needed. If the object has any holes, it should be fixed. This will ensure a good hand gesture recognition. Finally, the object inside the image is counted. If the number of the object is one (make sure that only the hand is detected in the image), then the algorithm can proceed further.

C. Finger Detection

First, the centroid of the binary image is determined. This will be the center of the hand which will be used to remove the wrist of the hand and palm. After finding the centroid of the image, the right side of the image which is the wrist will be removed as it does not involve the finger detection. Then, the largest distance between one pixel to another on the contour of the object is determined using the equidistance formula. The formula of equidistance is:[6]

$$dist = \sqrt{(y_2 - y_1)^2 + (x_2 - x_1)^2} \quad (3)$$

$$radius = dist \div 2 \quad (4)$$

where: (x_1, y_1) = Coordinate of a pixel

(x_2, y_2) = Coordinate of a pixel

The maximum distance will be divided by two to obtain the radius of the circle. A circle will be drawn in the image, and it will be removed (palm) as we obtain the radius and center of the circle. Moreover, the radius should times by 1.26 to make the circle bigger so that only the fingers can be obtained. The number of an object inside the image will be calculated and displayed [6]

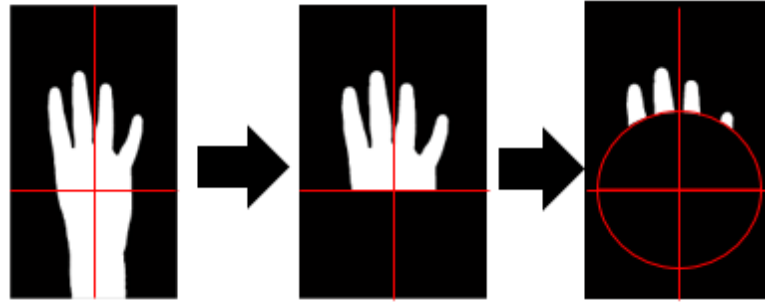


Figure 3.14 Finger detection for four finger counts

D. Gesture Classification

The detected fingers are classified into their respective finger count. The gestures are classified using the maximum distance between the centroid of the two fingers determined in the finger detection process. The distance between each centroid of the two fingers is determined using the equidistance Equation (3). [6]

3.13 TESTING AND ANALYSIS

This experiment is conducted with five subjects. For this experiment, a high definition camera was used to capture the image of hand gesture. This is because a low definition camera will produce more noise in the image which difficult to process. Fourteen hand gestures according to Figure 3 were used in this experiment, and 10 data were collected for each subject. Thus, a total of 700 data was collected in this experiment (14x5x10). [6]

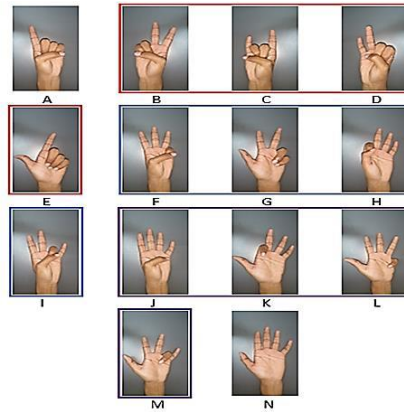


Figure 3.15 Hand gesture used in the experiment. (A and N uses one gesture to represent 1 and 5 finger counts, B to M uses four gestures to represent 2, 3 and 4 finger count)

In this experiment, one gesture was used for finger count one and five. Whereas, four gestures are used for finger count two, three and four. The images were taken in a controlled lighting environment (room light) to obtain a good result. The distance between the hand and the camera is between 15 to 20 cm. The data are collected for single hand recognition. Images are captured by an off-the-shelf HD webcam with a resolution of 16Mp to avoid noise in the image. Figure 5.4 shows the process flow of hand gesture recognition based on finger counting. [6]

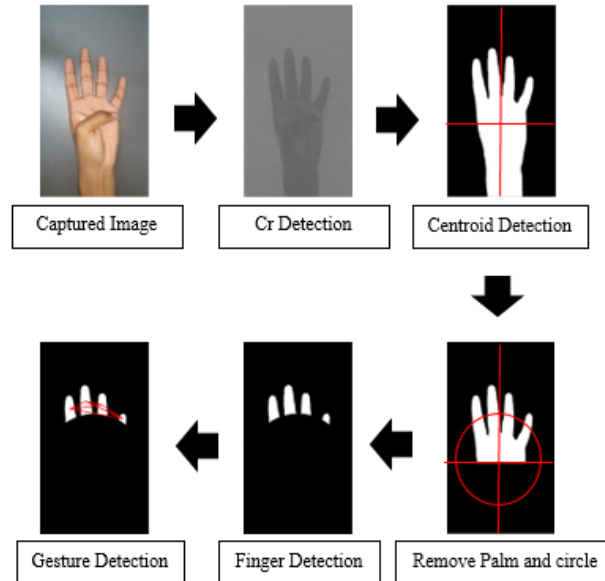


Figure 3.16 Graphical flow chart for hand gesture recognition algorithm based on finger counting

The algorithm was tested using different parameters to verify the performance of the hand gesture recognition algorithm. The parameters that chosen in this experiment were light intensity, size, noise, and effect of selective hand accessories. [6]

3.11 RESULT AND DISCUSSION

The experiment was carried out for one to five finger counts as shown in Figure 3.13. Five subjects were chosen for this experiment. The result was collected based on the number of successful hand gesture recognition for every finger count. Figure 3.14 shows the percentage of successful finger detection and gesture recognition for the algorithm that has developed. [6]

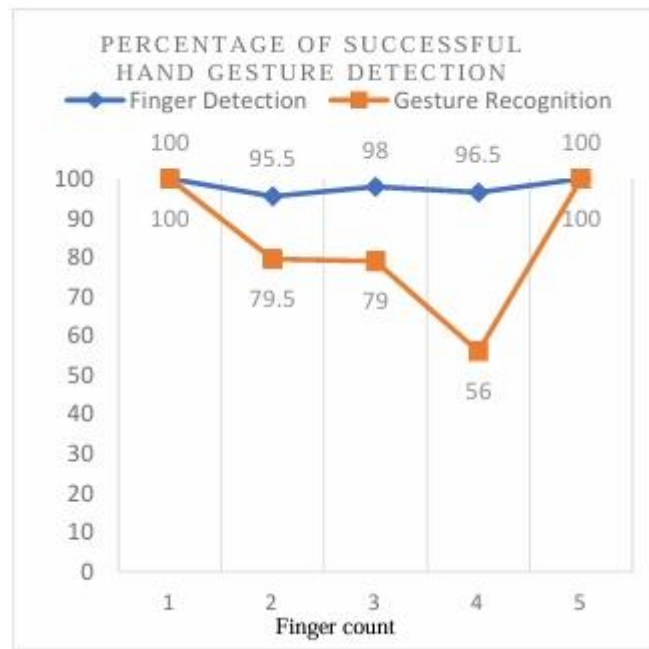


Figure 3.17 Percentage of successful hand gesture detection

The results show that the algorithm can correctly identify the number of fingers. This is indicated by high values of detection accuracies – more than 90% – in all cases for 1 to 5 fingers. The percentage of successful finger detection for finger count two is lower than the other finger counts because gesture 4 (refer Figure 5.3 image E) fail to detect correctly. When the circle is removed from the filtered image of finger count two for gesture 4, the thumb cannot be detected.

Additionally, the algorithm can produce 100% correct gesture recognitions for 1 and 5 finger count and good recognition (around 80% correct) for 2 and 3 finger counts. We also observed, the average (around 56%) gesture recognition accuracy for 4 finger counts. [6]

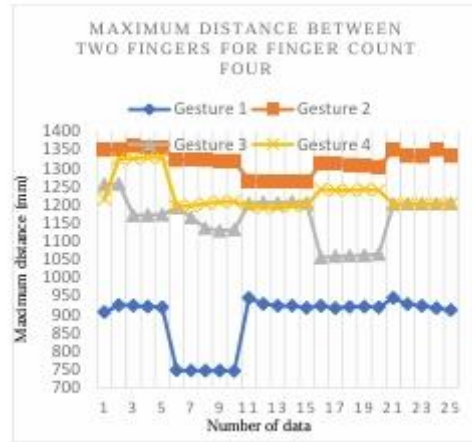


Figure 3.18 Maximum distance between fingers for four finger counts

Figure 3.18 shows that, 5 data of the maximum distance between fingers for four finger count which is collected from 5 subjects. It can be observed that gesture one and two can be easily classified as the value is far apart from one another. Whereas, there is an overlap for gesture three and four. This is mainly because the failure in finger detection which makes the maximum distance similar to other gestures. The performance of the hand gesture recognition algorithm was tested by varying some parameters in the algorithm. The parameters are noise, light intensity, size of the hand and effect of selective hand accessories. Figures 3.19 and 3.20 show the relationship between the successful hand gesture detection and the varied parameters.

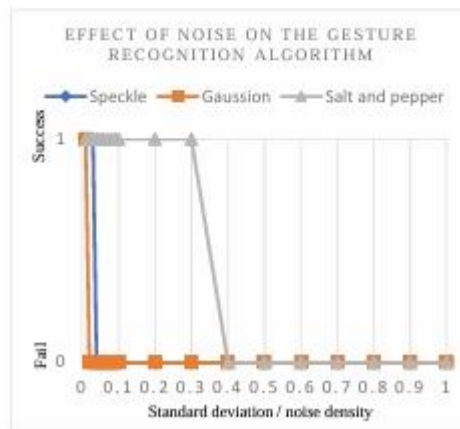


Figure 3.19 Effect of noise on the gesture recognition algorithm

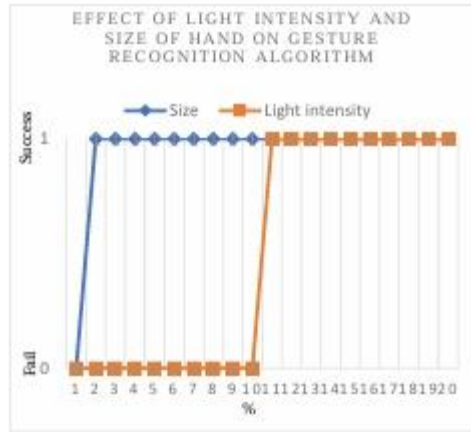


Figure 3.20 Effect of light intensity and size of a hand on the gesture recognition algorithm

A. Noise

Three types of noise were tested in this experiment, which are Gaussian noise, salt and pepper noise and speckle. We observed that, when the magnitude of noise in the image increases, it became more difficult to produce correct gesture recognition. For example, when salt and pepper noise with of more than 0.4 noise density were added as shown in Figure 9, the accuracy of the gesture recognition algorithm decreases by 50% due to the changes in the maximum distance between fingers. Additionally, for Gaussian noise, a mean of 1 and standard deviation more than 0.02 will reduce the accuracy of the algorithm by 90%. Finally, the accuracy of gesture recognition reduces by 90% when the standard noise deviation of more than 0.03 for speckle noise..[6]



Figure 3.21 Effect of salt and pepper noise at 0.4 noise density on hand gesture recognition algorithm

B. Light Intensity

The brightness of the image was changed to study the effect of light intensity on hand gesture recognition algorithm. If the brightness of the image is increased, the hand gesture algorithm

works perfectly. Whereas a decrease in the brightness of the image below 0.1% will decrease the percentage of successful hand gesture detection by 50%. The image cannot be detected if the brightness is below 0.01%.

C. Hand Size

The size of the image is changed using the MATLAB. The original image size is decreased to study the effect of hand size on hand gesture recognition algorithm. The gesture recognition reduces to 90% after the image is resized at a value of 0.01 from the original size of the image



Figure 3.22: Effect of size on the hand gesture recognition algorithm

D. Hand Accessories

We also have analysed the effect of common hand accessories such as watch and bracelet on gesture recognition performance. We observed that watch give a huge effect on the hand gesture recognition algorithm. The YCbCr color space detection fails to detect the silver watch. Thus, the watch separates the wrist and palm of the hand as shown in Figure 5.11. This is due to the limitation of using YCbCr color plane [6]

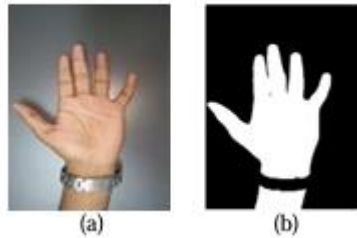


Figure 3.23 Effect of the watch on the hand gesture recognition algorithm (a) the original image and (b) the binary image

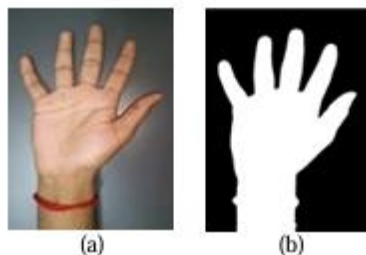


Figure 3.24 Effect of prayer thread on the hand gesture recognition algorithm (a) the original image and (b) the binary image

Chapter 4

Practical implementation and results

4.1 Introduction

Python is a high-level programming language widely used in Artificial Intelligence, Computer Vision, Automation, Robotics, and Data Analysis. Python is known for its simple syntax, readability, and powerful libraries that allow developers to create intelligent systems efficiently.

In this project, Python is used to develop an AI-based patient assistance system that detects hand gestures through a webcam and sends alerts using Telegram. The system helps patients communicate their needs without speaking, making it useful for elderly patients, disabled individuals, and patients with limited movement.

The project uses Computer Vision and Machine Learning technologies to analyze hand movements and recognize gestures. [3]

4.2 Hand Detection and Command Execution Flow

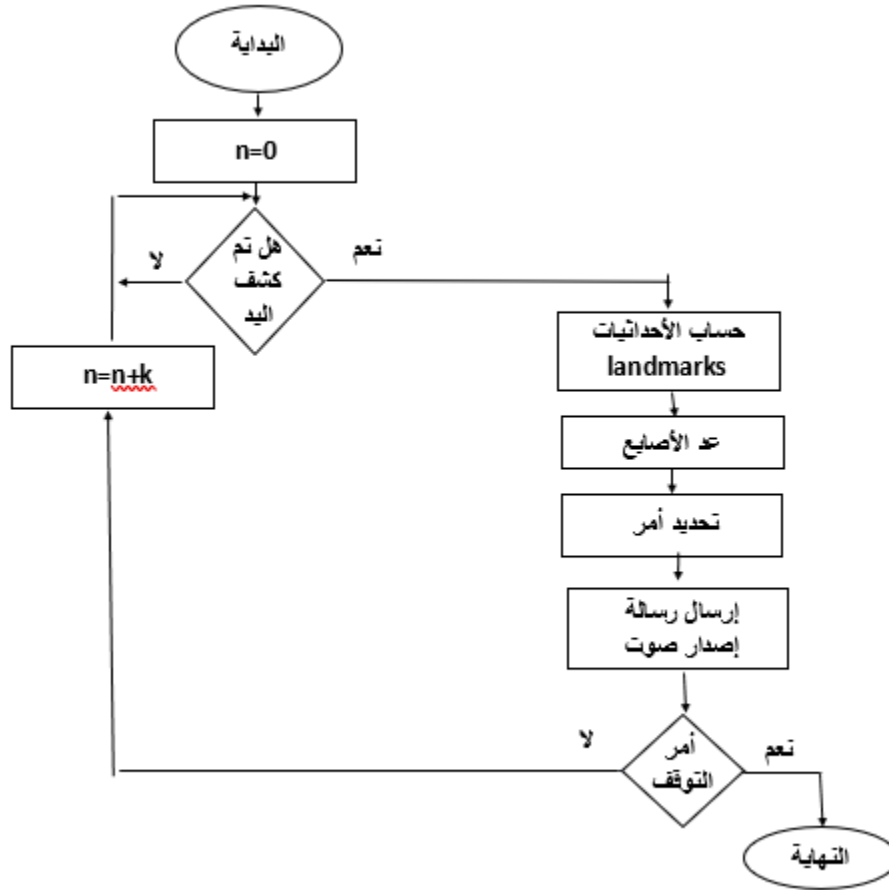


Figure 4.1 Flowchart for Printing Commands

This flowchart describes a real-time hand gesture detection and voice-command system. It's a continuous loop that keeps checking video frames for a hand, and whenever it finds one, it counts the fingers, maps that finger count to a specific command, and speaks that command aloud as an audio/voice message. This is very much like a "hands-free control system" — e.g., holding up 3 fingers might trigger the system to say "volume up" or similar, depending on however "تحديد أمر" (determine command) is defined in the full implementation.

Step 1: Start

The entry point of the program/system.

Step 2: $n = 0$

Initializes the frame counter to zero — this is frame number 0, the very first frame the system will examine.

Step 3: Decision — "Has a hand been detected?" (هل تم كشف اليد)

The system checks the current video frame (frame number n) to see if a hand is visible in it, typically using a hand-detection model.

Branch: "No" (لا) — no hand found in this frame

Control flows directly to $n = n + k$, skipping straight past all the processing steps. This makes sense: if there's no hand in the frame, there's nothing to analyze, so the system just advances the frame counter and loops back to check the next sampled frame.

Branch: "Yes" (نعم) — a hand was found in this frame

Control proceeds into the full processing pipeline:

Step 4: Compute landmark coordinates (حساب الإحداثيات landmarks)

The system extracts the specific coordinate points (landmarks) of the detected hand — these are the same kind of tracked points discussed in your previous flowchart (fingertip positions, knuckle positions, etc.), typically produced by a hand-tracking model like MediaPipe.

Step 5: Count the fingers (عد الأصابع)

This step is exactly the finger-counting algorithm from your earlier flowchart — it uses the landmark coordinates just computed to determine how many fingers are extended, producing a

count (this would be the "n" from your previous diagram, though note it's a completely different variable from the "n" used in *this* flowchart — the same letter is being reused for two unrelated purposes across the two diagrams, which is worth being careful about if you ever combine them into one system).

Step 6: Determine the command (تحديد أمر)

Based on the finger count from Step 5, the system maps that number to a specific predefined command. For example: 1 finger = one command, 2 fingers = another command, and so on. This is essentially a lookup/mapping step.

Step 7: Send a voice message (إرسال رسالة إصدار صوت)

The determined command is converted into an audio output — the system "speaks" or plays a sound corresponding to the recognized gesture. This could be via text-to-speech or a pre-recorded audio clip triggered by the command.

Step 8: Decision — "Stop command?" (أمر التوقف)

After issuing the voice message, the system checks whether the command that was just recognized/issued happens to be a special "stop" command (i.e., a gesture reserved for ending the program, like a specific finger count meaning "quit" or "shutdown").

Branch: "Yes" (نعم) — it was the stop command

Control proceeds to **End** — the program terminates.

Branch: "No" (لا) — it was a regular command, not the stop command

Control flows to $n = n + k$, advancing the frame counter, and then loops back up to Step 3 to check the next sampled frame for a hand again.

Step 9: $n = n + k$ (the merge/loop-back point)

Both "no hand detected" (Step 3) and "not a stop command" (Step 8) funnel into this same box. It advances the counter by k frames, then the flow returns to Step 3, restarting the cycle — checking the new frame (frame number n) for a hand.

Step 10: End

Reached only when the recognized gesture/command was specifically identified as the stop command. The continuous detection loop terminates here

4.3 Finger Status Evaluation and Counting Subsystem

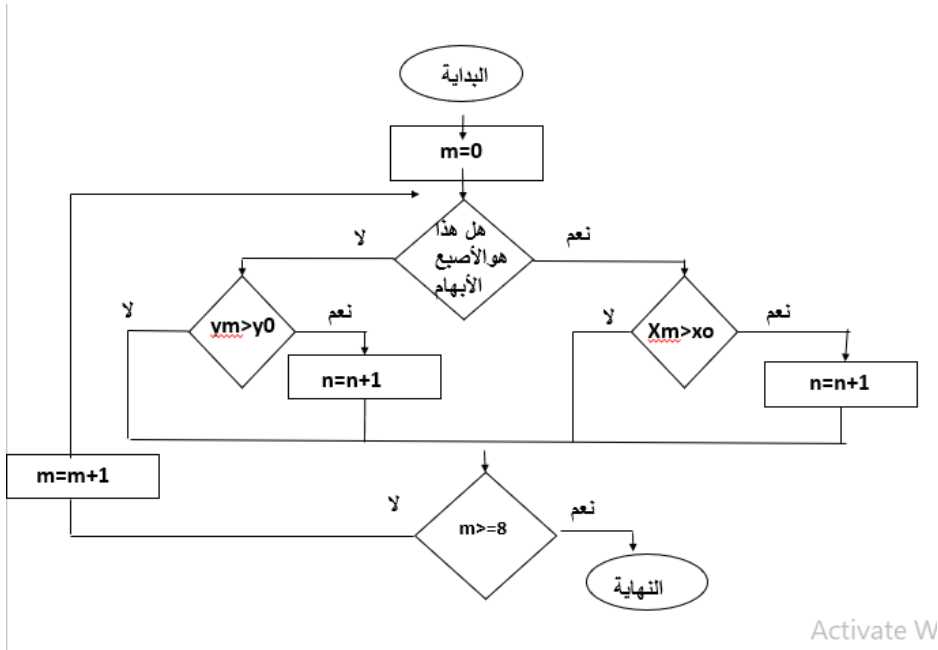


Figure 4. 2Finger Counting flowchart

. Start

- The process begins.

2. Initialization: m = 0

- m is set to 0. This is the index/counter representing which finger is currently being checked (finger #0 through #7).

3. Decision: "Is this the thumb finger?"

This is the key branching point — the algorithm treats the thumb differently from the other 4 fingers.

Branch A — "No" (لا): Not the thumb

- Go to the check: $Y_m > Y_0$ (Is the Y-coordinate of this finger greater than a reference value Y_0 ?)
 - **Yes (نعم)** → $n = n + 1$ (increment the counter — this finger is counted, e.g., "finger is up/raised")
 - **No (لا)** → skip counting, go directly down to the bottom loop check

Branch B — "Yes" (نعم): It IS the thumb

- Go to the check: $X_m > X_0$ (Is the X-coordinate of the thumb greater than a reference value X_0 ?)
 - **Yes (نعم)** → $n = n + 1$ (increment the counter — thumb is counted, e.g., "thumb is extended")
 - **No (لا)** → skip counting, go directly down to the bottom loop check

(Logic: since the thumb bends sideways rather than up/down, its "raised" state is measured by comparing X-position instead of Y-position, unlike the other four fingers.)

4. Both branches merge

- Whether or not n was incremented, both paths converge into the loop-control check.

5. Loop condition: $m \geq 8$

- **Yes (نعم)** → all 8 fingers have been checked → go to **End**
- **No (لا)** → not finished yet → go to $m = m + 1$

6. $m = m + 1$

- Move to the next finger index, then loop back up to step 3 ("Is this the thumb?") to repeat the check for the new finger.

7. End

- The loop terminates once all 8 fingers ($m = 0$ to 7) have been checked. At this point, n holds the total count of fingers that satisfied their respective condition (raised/extended fingers, for example — this looks like a **finger-counting / gesture-recognition algorithm**, likely used in hand-tracking, e.g., counting how many fingers are up

4.4 Conducted experiments

المسافة(cm)	الأضاءة	عدد التجارب	عدد مرات النجاح	عدد مرات الفشل	الضبط %
46(cm)	جيدة	50	42	8	82 %
46(cm)	منخفضة	50	37	13	74 %
75(cm)	جيدة	50	38	12	76 %
75(cm)	منخفضة	50	34	16	68 %
المجموع	-	200	151	49	76 %

$$accuracy = \frac{\text{عدداختبارات الناجحة}}{\text{اختبارات الكلية}}$$

1. High Performance under Optimal Conditions The system achieved its highest accuracy at a distance of **46 cm** under good lighting conditions (**84%**, with 42 successful attempts out of 50). This is attributed to the clarity of the target features and minimal visual noise in the captured images.

2. Impact of Low Lighting at Close Distance When the lighting was reduced at the same distance (**46 cm**), the accuracy dropped from **84%** to **74%** (a 10% decline). This demonstrates that low light impacts image contrast and feature details, leading to an increase in failures (13 failed attempts).

3. Impact of Increased Distance with Good Lighting Increasing the distance to **75 cm** while maintaining good lighting resulted in an accuracy of **76%** (38 successful attempts out of 50). This indicates that greater distance reduces the pixel resolution of the target region in the camera frame, though adequate lighting helps maintain an acceptable performance level above 75%.

4. Performance Degradation under Adverse Conditions The lowest accuracy was recorded when combining both negative factors: a long distance (**75 cm**) and low lighting, where

accuracy fell to **68%** (16 failed attempts out of 50). This highlights the cumulative effect of reduced pixel resolution and poor contrast, making feature recognition more challenging for the model.

5. Overall Project Reliability

- **Total Experiments:** 200 tests conducted across 8 commands to ensure a statistically sound sample size.
- **Total Successes:** 151 successful attempts out of 200.
- **Overall Accuracy:** 75.5% (rounded to **76%** in official reporting).
- **Conclusion:** The project is reliable and operates successfully above the target threshold (70%), displaying a logical performance trend across varying environmental conditions

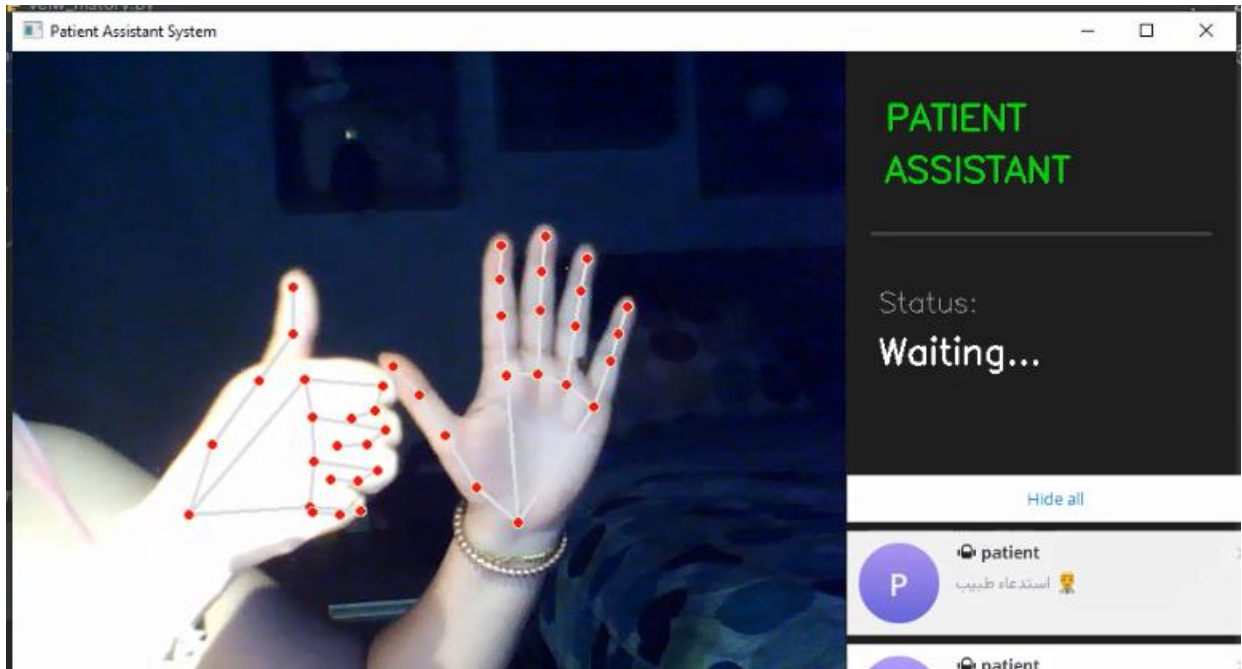


Figure 4.3 patient assistant system window

system Interface

The system interface was designed to be simple, user-friendly, and easy to monitor during operation. It is divided into two main sections: the **camera view**, where the patient's hand gestures are captured and analyzed, and the **control panel**, which displays the system status and the transmitted requests.

1. Camera View

The camera view occupies the largest area of the interface. It displays the live video stream captured by the camera in real time, allowing the patient to perform hand gestures naturally.

When the system detects one or both hands, it overlays **red landmark points** connected by lines on the detected hands. These landmarks represent the joints and key positions of the fingers and palm, enabling the system to monitor hand movements accurately. Displaying these landmarks allows the user to verify that the hands have been detected correctly before gesture recognition takes place.

The camera view is continuously updated, ensuring smooth real-time interaction between the patient and the system.

2. System Information Panel

The panel on the right side of the interface provides information about the current operation of the system.

At the top of the panel, the system name "**Patient Assistant**" is displayed to identify the application.

Below the title, the **Status** section indicates the current operating state of the system. For example, the message "**Waiting...**" means that the system is active and waiting for the patient to perform a valid hand gesture. This status changes automatically according to the system's operation, helping the user understand what the system is currently doing.

3. Sent Messages Panel

The lower section of the interface displays the **Sent** panel, which contains all requests successfully recognized and transmitted by the system.

Whenever a patient performs a valid gesture, the corresponding request is immediately sent and displayed in this panel. Each recognized request appears as a separate message, allowing medical staff to review the patient's latest requests easily.

Examples of transmitted requests include:

- Calling a doctor.
- Requesting water.
- Requesting medication.
- Emergency assistance.

This feature provides a clear history of the communication between the patient and the medical staff.

4. Overall Interface Operation

All components of the interface operate simultaneously. While the camera continuously captures the patient's hand movements, the system analyzes the gestures in real time, updates the system status, and displays recognized requests immediately after they are sent.

This integrated design enables fast, reliable, and intuitive communication between the patient and healthcare providers, ensuring that patient requests are delivered efficiently and can be monitored at any time

The screenshot displays the 'Clinical Decision Support' window for patient ID 101. The window title is 'سجل وتقارير المريض الطبية - Clinical Decision Support'. The main heading is 'تقرير متابعة المريض: سدره (ID: 101)'. Below this is a table with two columns: 'الوقت والتاريخ' (Time and Date) and 'نوع الطلب / الإيماءة' (Request Type / Gesture). The table lists eight requests with their corresponding times and dates. Below the table is a section titled 'التحليل الذكي والتوصيات الطبية (آخر 8 ساعات - الشفت الحالي)' (Smart Analysis and Medical Recommendations (Last 8 hours - current shift)). This section contains four bullet points with icons: a warning triangle for 'مؤشر تخفيف: تكرار طلب (الشعور بالجوع) 26 مرات خلال المناوبة الحالية (8 ساعات). يُوصى ببدائل غذائية**' (Warning: Repeat request (feeling hungry) 26 times during the current shift (8 hours). Dietary alternatives are recommended**), a warning triangle for 'مؤشر ألم مرتفع: تكرار طلب (ألم/طبيب) 69 مرات خلال المناوبة. يُوصى بتقييم المسمكن**' (Warning: High pain indicator: Repeat request (pain/doctor) 69 times during the shift. Analgesic evaluation is recommended**), a bed icon for 'عدم راحة جسدية: طلبات متكررة لتعديل السرير خلال المناوبة. يُوصى بتغيير وضعية المريض**' (Physical discomfort: Repeated requests to adjust the bed during the shift. Patient position change is recommended**), and a person icon for 'دعم نفسي مطلوب: ارتفاع إشارات القلق أو طلب الأهل خلال المناوبة الحالية**' (Psychological support needed: Increased anxiety signals or family requests during the current shift). At the bottom left, it says 'إجمالي الطلبات (خلال الـ 8 ساعات الأخيرة): 252'. At the bottom right, there is a red button labeled 'إغلاق التقرير' (Close report).

الوقت والتاريخ	نوع الطلب / الإيماءة
2026-08-07 02:06:19	حالة نفسية: ملل
2026-08-07 02:06:16	طلب عائلته / رؤية الأهل
2026-08-07 02:06:12	حالة نفسية: ملل
2026-08-07 02:06:08	السرير غير مريح / تعديل وضعية
2026-08-07 02:06:05	حالة نفسية: ملل
2026-08-07 02:06:02	يوجد ألم / شعور بالألم
2026-08-07 02:05:59	طلب عائلته / رؤية الأهل
2026-08-07 02:05:54	حالة نفسية: ملل

التحليل الذكي والتوصيات الطبية (آخر 8 ساعات - الشفت الحالي)

- مؤشر تخفيف: تكرار طلب (الشعور بالجوع) 26 مرات خلال المناوبة الحالية (8 ساعات). يُوصى ببدائل غذائية**
- مؤشر ألم مرتفع: تكرار طلب (ألم/طبيب) 69 مرات خلال المناوبة. يُوصى بتقييم المسمكن**
- عدم راحة جسدية: طلبات متكررة لتعديل السرير خلال المناوبة. يُوصى بتغيير وضعية المريض**
- دعم نفسي مطلوب: ارتفاع إشارات القلق أو طلب الأهل خلال المناوبة الحالية**

إجمالي الطلبات (خلال الـ 8 ساعات الأخيرة): 252

إغلاق التقرير

Figure 4.4 Clinical Decision Support (CDS) window

This is a **Clinical Decision Support (CDS) window** — a patient monitoring dashboard built with Python (using tkinter for the GUI and sqlite3 for the database). It logs a patient's requests/gestures over time and automatically analyzes them to flag potential care needs.

Window Structure (top to bottom)

1. Title Bar

- "سجل وتقارير المريض الطبية - Clinical Decision Support" — the patient's log and .medical report window

2. Header Label

- Shows the specific patient being monitored: "(ID: 101) تقرير متابعة المريض: سدره" (Patient Follow-up Report: Sedra, ID 101) — pulled dynamically from the patient_name and patient_id parameters passed into show_log_window().

3. Data Table (Treeview)

Two columns:

- نوع الطلب / الإيماءة (Request Type / Gesture) — e.g "طلب رؤية الأهل" (boredom), "يوجد ألم / شعور بالألم" (uncomfortable bed / reposition), "حالة نفسية: ملل" (psychological state:)
- الوقت والتاريخ (Date) — timestamp of each logged request & Time)

This table is populated from the SQLite database, filtered by a **rolling time window**.

4. The "Every 8 Hours" Logic (core feature)

This is the key mechanic you asked about:

- The code defines `FIXED_WINDOW_HOURS = 8` at the top, meant to align with a typical **nursing/doctor shift duration**.
- Every time the report window opens, it calculates `time_limit = now - 8 hours` and queries the database for only requests logged **within the last 8 hours** — i.e., within the current shift.
- This means the dashboard doesn't show the patient's *entire* history — only what's relevant to the **current shift**, so incoming staff can quickly assess the situation without digging through old data.
- The panel title literally reflects this: "التحليل الذكي والتوصيات الطبية (آخر 8 ساعات - الشفت الحالي)" — "Smart Analysis & Medical Recommendations (Last 8 hours – current shift)."

- The footer also reflects the same window: "إجمالي الطلبات (خلال الـ 8 ساعات الأخيرة): 252" — total requests within the last 8 hours.

5. Smart Analytics Panel (generate_clinical_insights)

This function scans all requests within that 8-hour window and generates automatic clinical flags based on thresholds:

Category	Trigger condition	Insight generated
 Nutrition	"hunger"/"food" requests ≥ 3 times	Recommends food alternatives
 Pain	"pain"/"doctor" requests ≥ 2 times (lowered because window is 8h)	Recommends painkiller evaluation
 Physical comfort	"bed"/"comfort" requests ≥ 3 times	Recommends repositioning patient
 Psychological	"anxiety"/"family" requests ≥ 2 times	Recommends psychological support

If none of these thresholds are hit, it defaults to: "✓ حالة المريض مستقرة" (patient's condition is stable).

In the screenshot, you can see this working live — it flagged a **high hunger indicator (26 times)** and a **high pain indicator (69 times)** within the current 8-hour shift, each with a specific recommendation.

6. Footer

- Left: total request count in the current 8-hour window.
- Right: a red "إغلاق التقرير" (Close Report) button that closes the window.

References

- [1] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. New York, NY, USA: Pearson, 2018.
- [2] S. Borman and R. Stevenson, "Spatial Resolution Enhancement of Low-Resolution Image Sequences: A Comprehensive Review," Dept. Electrical Engineering, Univ. Notre Dame, Notre Dame, IN, USA, Tech. Rep., Oct. 2002.
- [3] F. Zhang et al., "MediaPipe Hands: On-device Real-time Hand Tracking," arXiv preprint arXiv:2006.10214, 2020.
- [4] B. Sahu and K. Rajpoot, "Real-Time Hand Finger Recognition and Counting (Using OpenCV and MediaPipe)," 2026
- [5] A. Swarnakar, A. Kanade, A. Pal, S. K. Soni, and R. Kumar, "Hand gesture recognition and finger count," International Journal of Engineering Research & Technology (IJERT), ICSITS 2020, vol. 8, no. 5, Mar. 2020
- [6] M. Perimal, S. N. Basah, M. J. A. Safar, and H. Yazid, "Hand-gesture recognition-algorithm based on finger counting," Journal of Telecommunication, Electronic and Computer Engineering (JTEC), School of Mechatronic Engineering, Universiti Malaysia Perlis, vol. 10, no. 1–13, pp. 19–24, 2018